



# Data Warehouses

Lecture 11

# Lecture Goals

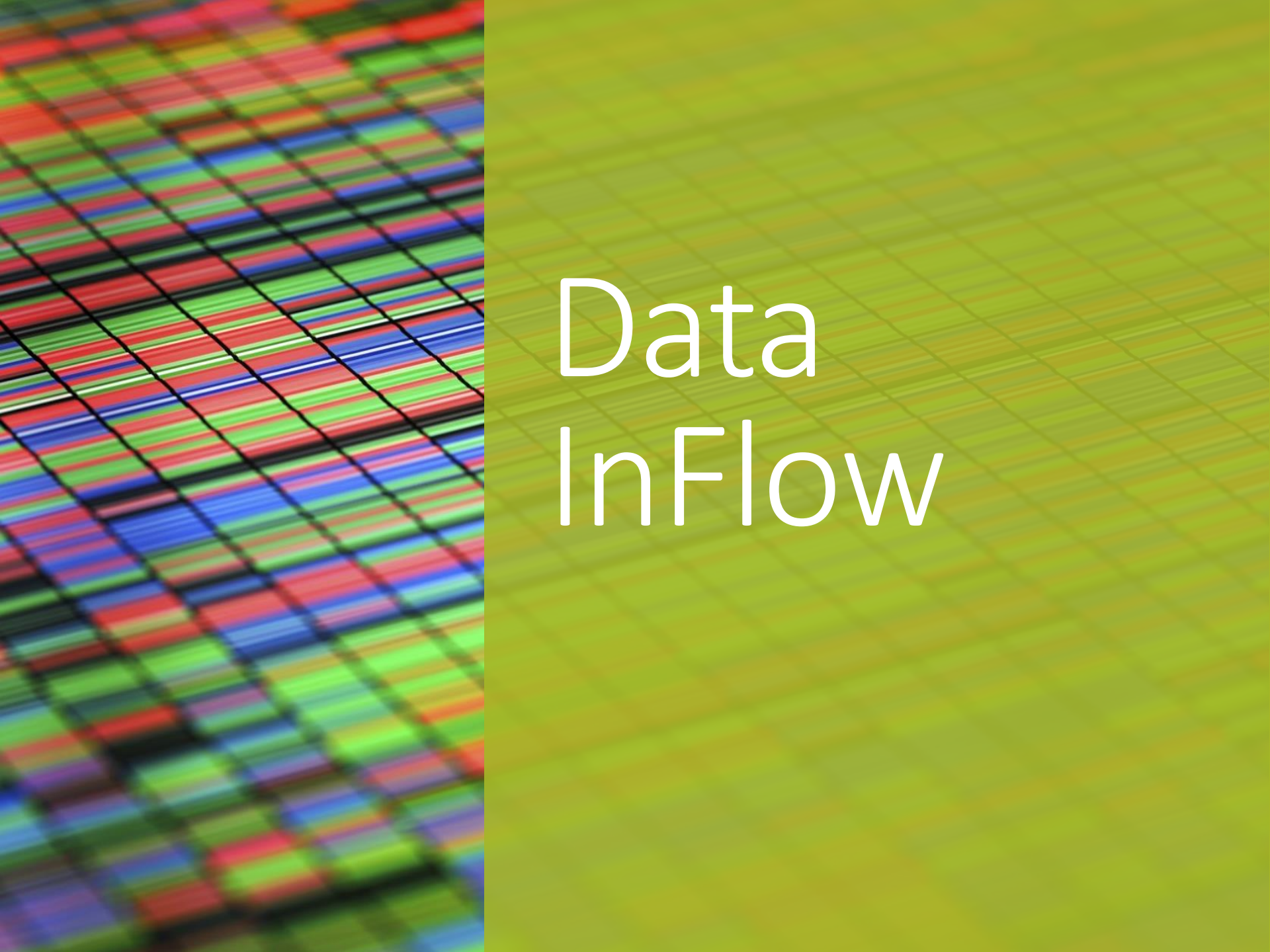
ETL

*E – extract*

*T – transformation*

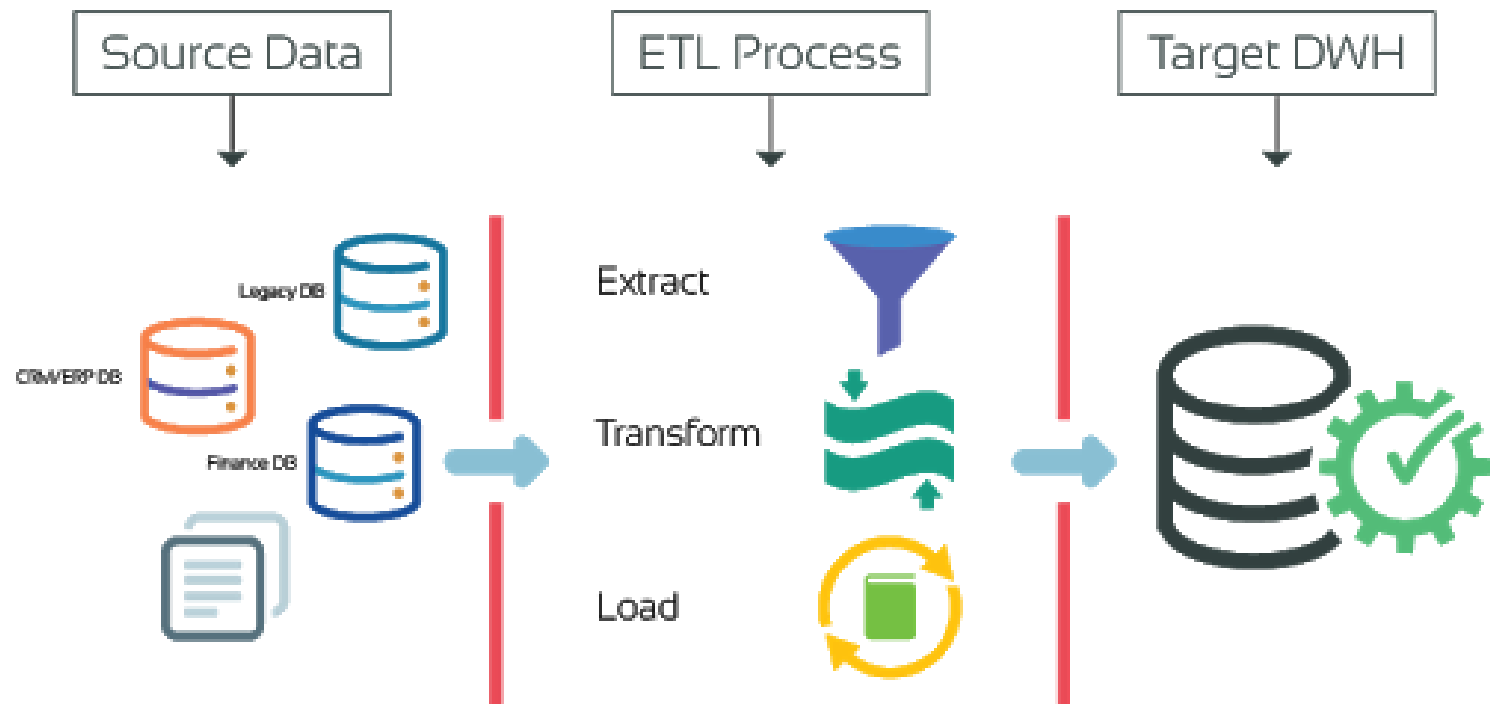
Clean and Conform

*L – load*



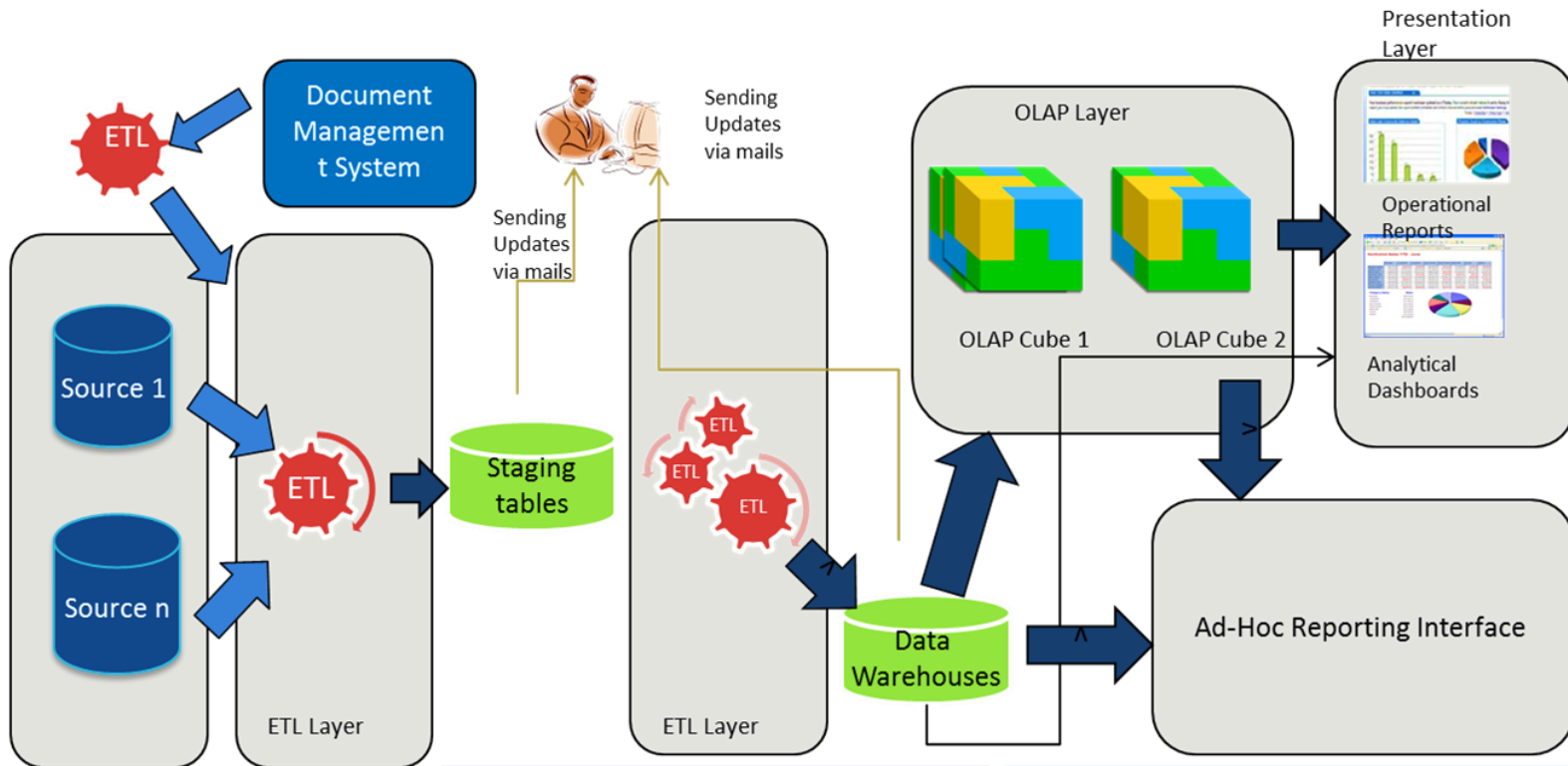
# Data InFlow

# InFlow

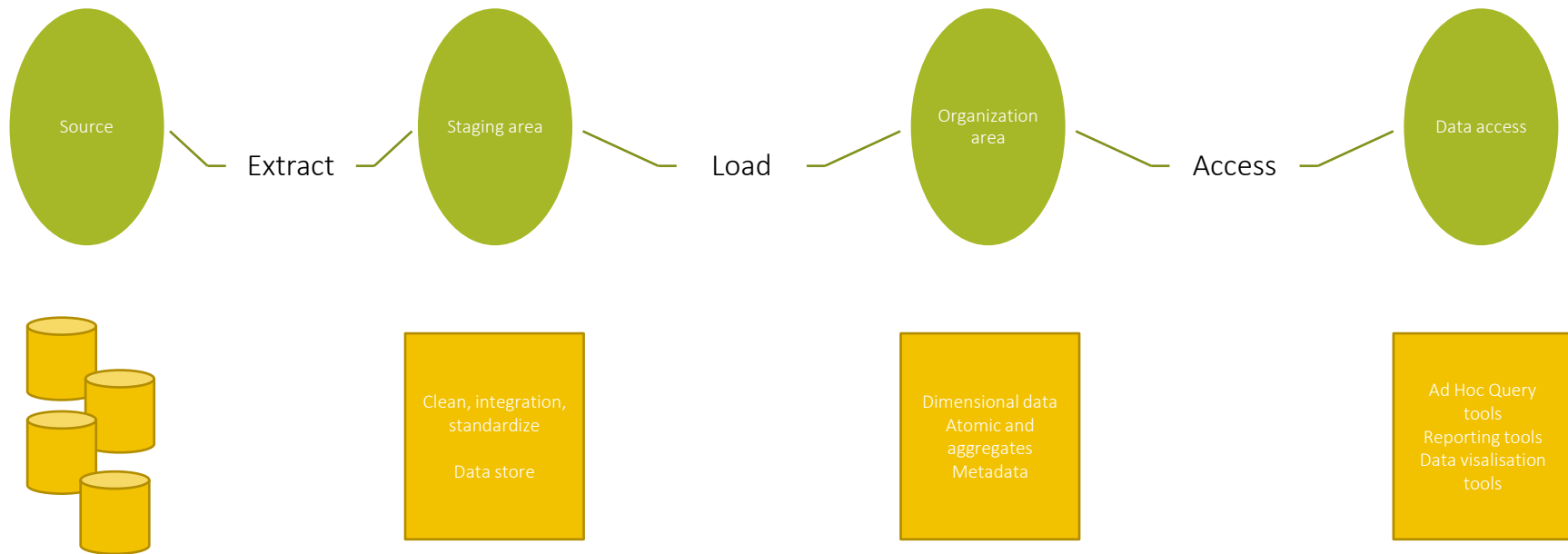


# Data Warehouse

Data warehousing



# Data Warehouse



# ETL

*A properly designed ETL system  
**extracts data** from the source systems,  
**enforces data quality and consistency** standards,  
**conforms data** so that separate sources can be used together,  
and finally **delivers data** in a presentation-ready format  
so that application developers can build applications and end  
users can make decisions*

R. Kimball, 2004





# Why TO ETL

First impressions, though, can be very deceiving:  
appears to be nothing more than the movement  
of data from one place to another



# Why pre-process Data ?

No quality data, no quality analysis !

Data warehouse needs consistent integration of quality data

*Data extraction, cleaning, and transformation comprises the majority of the work of building a data warehouse*

B. Inmon

Data is never perfect

*The cost of perfection, extremely high vs. its value.*

*the objective should be to get as clean data as can be based on the given constraints*

# Why pre-process Data ?

Why we care about ETL?

Data from different source systems will be different, poorly documented and dirty.

*Lot of analysis is required*

*E.g. – Easy to collate addresses and names?*

Not really. No address or name standards.

Use software for standardization.

Very expensive, as any “standards” vary from country to country, not large enough market.

Manual data collection and entry

*Nothing wrong with that but has potential to introduce lots of problems.*

# Why pre-process Data ?

Data in the real world is dirty

incomplete:

*lacking attribute values, lacking certain attributes of interest, or containing only aggregate data*

e.g., occupation=""

noisy:

*containing errors or outliers*

e.g., Salary="-10"

inconsistent:

*containing discrepancies in codes or names*

e.g., Age="42" Birthday="03/07/1997"

e.g., "Was rating "1,2,3", now rating "A, B, C"

e.g., discrepancy between duplicate records

Data in the real world is dirty

Incomplete data comes from

*n/a data value when collected*

*human/hardware/software problems*

Noisy data comes from the process of data

*collection*

*entry*

*transmission*

Inconsistent data comes from

*Different data sources*

*Functional dependency violation*

# Why pre-process Data ?

## “Some” Issues

Diversity in source systems and platforms

*Dozens of source systems across organizations*

Inconsistent data representations

*Same data, different representation*

*Multiple sources for same data element*

Complexity of transformations

*Simple one-to-one scalar transformations.*

*One-to-many element transformations.*

*Complex many-to-many element transformations.*

Rigidity and unavailability of legacy systems

*Very difficult to add logic to or increase performance of legacy systems*

Volume of legacy data

*Talking about not weekly data, but data spread over years.*

# Why pre-process Data ?

## General idea

Data is cleansed as it passes from the operational environment to the data warehouse environment.

*In some cases, a simple algorithm is applied to input data in order to make it correct. In complex cases, artificial intelligence subroutines are invoked to scrub input data into an acceptable output form.*

*There are many forms of data cleansing, including domain checking, cross-record verification, and simple formatting verification.*

Multiple input sources of data exist and must be merged as they pass into the data warehouse

*Logic must be spelled out to have the appropriate source of data*

Default values must be supplied

ETL is a recurring activity (daily, weekly, monthly)

# Why pre-process Data ?

## Context

- Deep historical data
- Important

## Consolidation

- All data in a single place
- Perfect for data analysis

## Productivity

- Doing all this manually
- Repeatable process

## Accuracy

- Good quality of data
- Consistent definitions



ETL in short

# ETL

The Extract-Transform-Load (ETL)  
is the foundation of the data warehouse.

Although building the ETL system is a back room activity  
*is not very visible to end users*

it easily consumes 70 percent of the resources  
*all resources !*

time, money, staff, processing power, etc.

it is the most time-consuming process in DW development  
*80% of development time spent on ETL*

# ETL

In short

The ETL **makes or breaks** the data warehouse.

A properly designed ETL:

- extracts data from the source systems,
- enforces data quality and consistency standards,
- conforms data so that separate sources can be used together,
- and finally delivers data in a presentation-ready format

*so that application developers can build applications and end users can make decisions.*

# ETL

## Problems

1. Data from different sources
2. Data with different formats
3. Handling of missing data and erroneous data
4. Query performance of DW

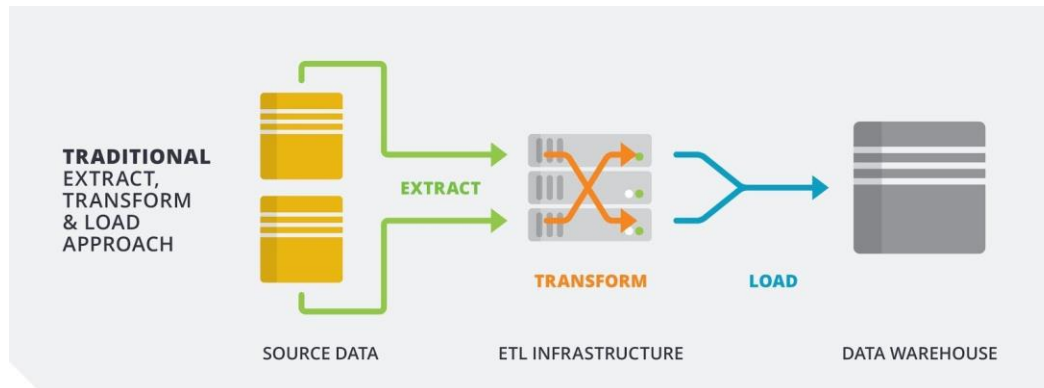
## Extract, Transform, Load (ETL)

“Getting multidimensional data into the DW”

*Extract (for problem #1)*

*Transformations / cleansing (for problems #2, #3)*

*Load (for problem #4)*



# ETL

## Extraction

refers to pulling the source data from the original database or data source.

## Transformation

refers to the process of changing the structure of the information, so it integrates with the target data system and the rest of the data in that system.

## Loading

refers to the process of depositing the information into a data storage system.

A continuous, ongoing process with a well-defined workflow

- first extracts data

- deposits the data into a staging area.

- the data goes through a cleansing process, gets enriched and transformed,

- is finally stored in a data warehouse

# ETL

## ETL:

Data extraction

*get data from multiple, heterogeneous, and external sources*

Data cleaning

*detect errors in the data and rectify them when possible*

Data conforming

*convert data from legacy or host format to warehouse format*

Load

*sort, summarize, consolidate, compute views, check integrity, and build indices and partitions*

Refresh

*propagate the updates from the data sources to the warehouse*

# ETL

## Extraction

Interface data in source systems

*(wrappers, monitors, translators, extractors)*

*Different classes of information sources*

*Change detection*

## Data discovery

*Collecting and tracking source systems*

Determining the system-of-record (the originating source of data)

Analyzing the source system to obtain important characteristics

## Anomaly detection

# ETL

## Transformation

Cleaning and conforming are the main steps where the ETL system adds value.

*The other steps of extracting and delivering are obviously necessary, but they only move and reformat data.*

*Cleaning and conforming actually changes data and provides guidance whether data can be used for its intended purposes*

*Often up about 80% of effort*

consists of

*removing all inconsistencies and redundancies of data coming to the data warehouse from operational data sources*

*conforming to the conceptual schema used by the warehouse.*

## Cleaning

*Improve data quality – data screening*

## Conforming

*Data integration – combining information into comprehensive views*



# ETL

## Transformation

### Cleansing

*Discard or correct erroneous data, e.g. attribute value that do not match, patch missing or unreadable data*

*Insert default values*

*Eliminate duplicates and inconsistencies, e.g. purchases of the customer who does not exist*

### Integration

*Merging, splitting and standardization*

Aggregate

Summarize

Sample

*Coding with common format*

*Splitting attribute values into several new values, e.g. split “address” into “zip code” and “city”*

# ETL

After extracting, cleaning and transforming, data must be loaded into the warehouse.

Loading the warehouse includes some other processing tasks:

*checking integrity constraints, sorting, summarizing, creating indexes, etc.*

Batch load utilities are used for loading.

A load utility must allow the administrator to monitor status, to cancel, suspend, and resume a load, and to restart after failure with no loss of data integrity

Loading

Initial loading

View maintenance (Warehouse data  $\approx$  materialized view)

# Some history

## The Birth of Data Engineering

Data engineering as a practice has existed in some form since companies started doing things with data.

*Before it came into sharp focus as a distinct field alongside the rise of data science in the 2010s, the practice had been branded in a whole host of different ways in the past, including as Database Administration, Data Analysis, Business Intelligence Engineering, Database Development, and more.*

The birth of Data Engineering can arguably be traced back to data warehousing

To move data from OLTP systems (the current state) to OLAP systems (the historical data), ETL (Extract, Transform, Load) processes emerged.

# Some history

## The Rise of ETL

Data engineering roots trace back to the emergence of data warehousing in the 1970s and 1980s.

By the 1990s, as enterprises sought to consolidate data from various operational systems (OLTP) into systems capable of analytical processing (OLAP), the need for a structured integration process became clear.

The term "ETL" was formalized in 1992 to describe this framework.

# Some history

In the traditional ETL model:

- Extract: Data is gathered from various source systems.

- Transform: Data is cleaned, standardized, and modified in a dedicated staging area or processing engine.

- Load: The finalized, "clean" data is loaded into the target data warehouse.

This approach was designed for an era of limited computing power and expensive storage.

- Because resources were constrained, it was necessary to transform data before storing it to keep the warehouse efficient.

# Some history

## The Shift - Limitations and the Transition to ELT

While ETL served as the primary method for data processing for decades, the evolution of cloud technologies led to the rise of Extract, Load, Transform (ELT) as a modern alternative.

*As data volumes grew and cloud technologies matured*

marked by the release of cloud-native, massively parallel processing (MPP) warehouses like Amazon Redshift in 2012

the traditional ETL model faced challenges:

*Inflexibility: Any change in business requirements necessitated time-consuming updates to the upstream transformation code.*

*Latency: The heavy upfront transformation processing created bottlenecks, delaying data availability.*

*Complexity: Managing staging areas and brittle, hard-coded pipelines often resulted in high maintenance overhead.*

# Some history

## The Modern Era: The Rise of ELT

The advent of the "Modern Data Stack" enabled the shift to ELT.

*With the cloud offering elastic storage and compute, it is no longer necessary to transform data before it hits the warehouse.*

In the ELT model, the sequence changes:

*Extract: Data is collected from sources.*

*Load: Raw data is loaded directly into the destination (data warehouse or data lake) immediately.*

*Transform: Transformation happens inside the target platform using its native processing power.*

This evolution has fundamentally changed the role of data engineers.

*By deferring transformation, organizations gain greater flexibility, faster time-to-insight, and simpler, more modular pipelines that can easily adapt to changing business needs.*

# Some history

## The Future of Data Engineering and ELT

As more organizations shift towards cloud-based infrastructures

*ELT will continue to play a crucial role in data integration processes*

ELT tools will continue to mature

*extending their coverage to more use cases to become more reliable foundational technologies*

Core aim will remain the same:

*to reduce complexity and increase modularization*

Popularity of streaming transformations

*Batch transformations are overwhelmingly popular*

*growing popularity of stream-processing solutions*

Reverse ETL

*As the analytics technology has progressed, enterprises unlock insights from their data that were not available before.*

*Recent trend indicates to treat the data warehouse as another operational data store that pushes the data generated by those insights back into the operational sources the ETL process depends on*



# ELT

## ELT

stands for extract, load, and transform.

It's a data pipeline model where data is first extracted from various source systems.

Then, instead of being transformed in a separate staging area, the raw data is directly loaded into a target data store, such as a data lake or a cloud data warehouse.

Only after the data is loaded into the target system are the transformations applied.

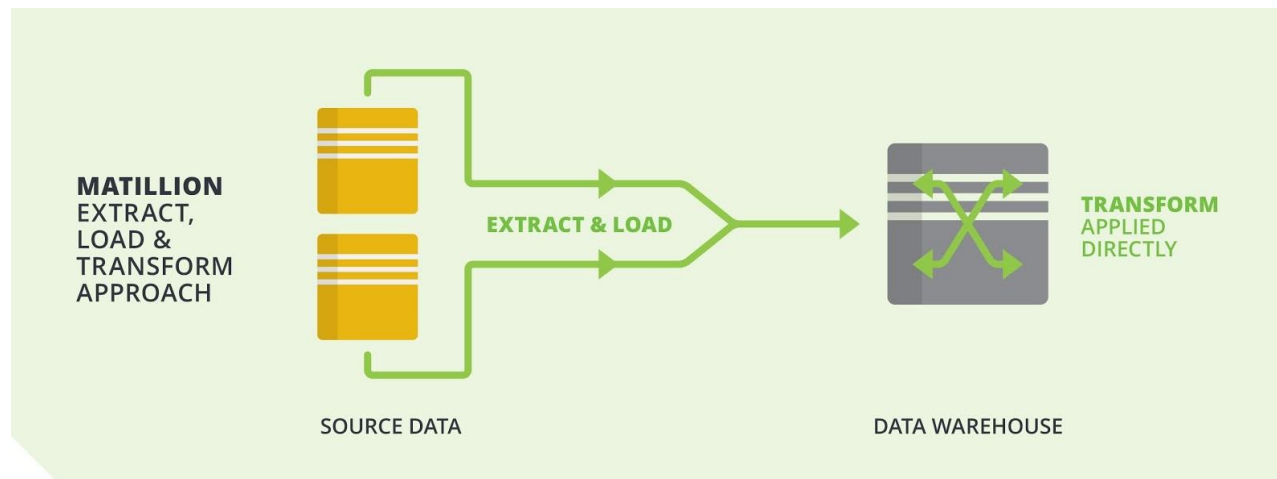
# ELT

## ELT

Extract and Load are done in one move

Data is extracted straight to target data platform.

Transformations are then applied on target data platform.



# ELT

The ELT process flow capitalizes on the power and scalability of modern data storage and processing platforms. Let's break down each component:

**Extract:** This initial step involves collecting raw data from its original sources. These sources can be incredibly diverse, including databases (SQL and NoSQL), enterprise applications (like CRMs and ERPs), SaaS platforms, APIs, and log files. The extraction process focuses on getting the data out of these systems efficiently.

**Load:** In the second step, the extracted raw data is loaded, often in its original format or with minimal processing, directly into a high-capacity storage system. Common targets for this raw data are cloud data lakes or modern cloud data warehouses that can handle large volumes of structured, semi-structured, and unstructured data.

**Transform:** This final step occurs after the data is safely housed in the target system. Using the computational power of the data warehouse or data lake, the raw data is cleaned, structured, enriched, and converted into a format suitable for analytics, reporting, and machine learning. Transformations can include filtering, joining, aggregating, standardizing formats, and deriving new data points.

The ELT process offers flexibility because transformations are not fixed before loading. Data scientists, for instance, can access the raw data to explore unforeseen patterns or conduct ad-hoc analyses, while business intelligence teams can build curated, transformed datasets for reporting.

# ELT

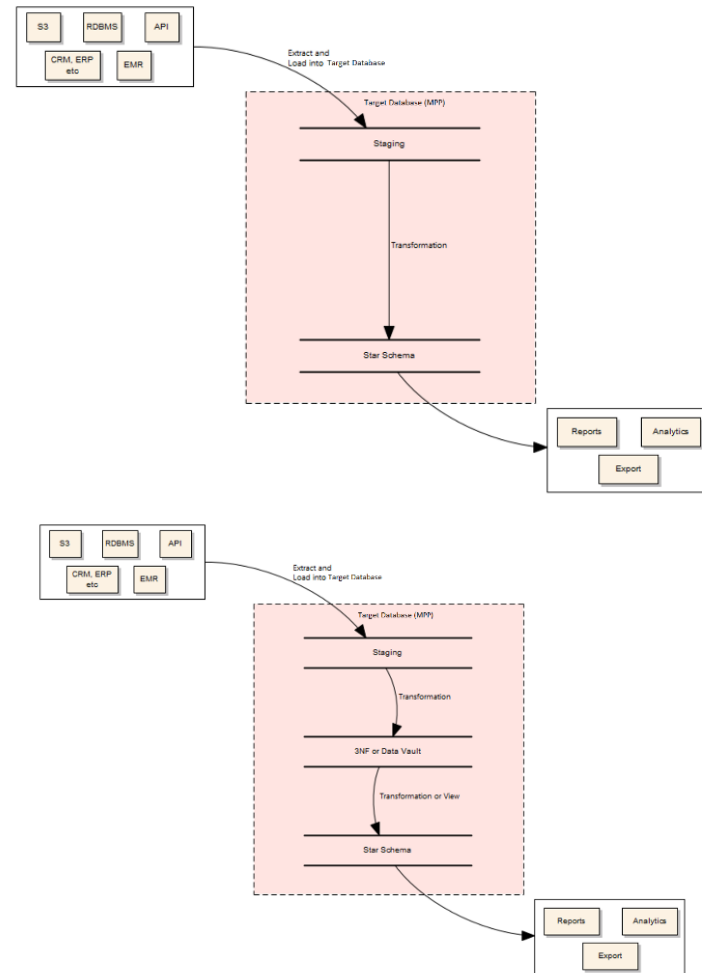
## ELT

Instead of transforming your data in your ETL system

*use the power of data warehouse to process transformations once loaded*

Data then becomes available in the database (staging area) or separate schema for raw data ready to do something with downstream.

The final stage sends transformations into the Star Schema.



# Medallion Data Zoning



## Bronze (Raw)

Permanent record of source data. No transformations applied.

## Silver (Cleaned)

Deduplicated, type-casted, and standardized. Ready for data science.

## Gold (Business)

Star-schemas and aggregations ready for BI tools.

Bronze



Raw

Silver



Validated

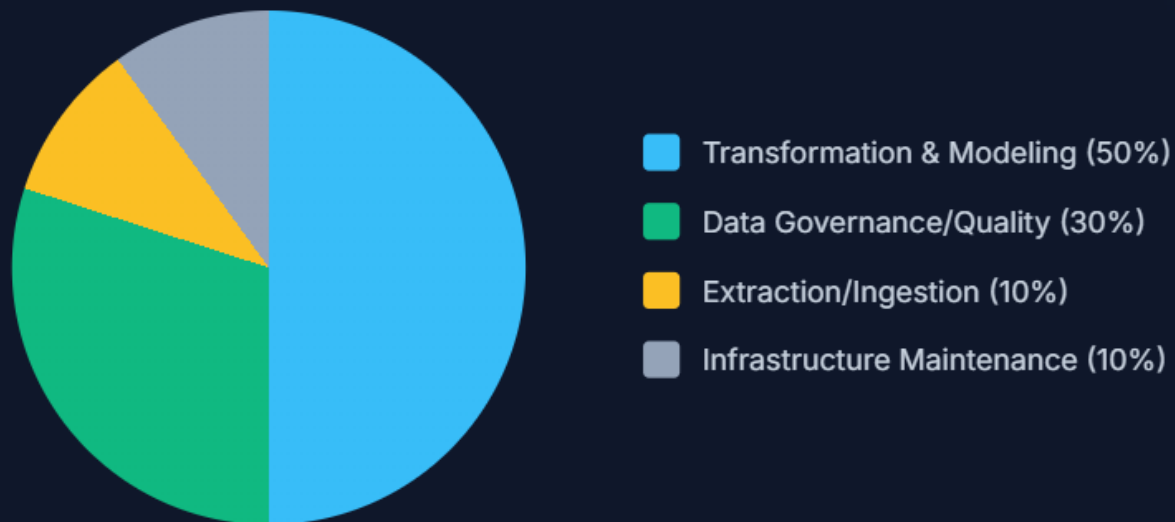
Gold



Enriched

# ELT

## Resource Allocation in ELT



# ETL vs ELT

| Feature                  | ELT (extract, load, transform)                                   | ETL (extract, transform, load)                          |
|--------------------------|------------------------------------------------------------------|---------------------------------------------------------|
| Order of operations      | Extract, then load, then transform                               | Extract, then transform, then load                      |
| Transformation location  | Within the target data store (data warehouse/lake)               | In a separate staging area or ETL tool environment      |
| Data loaded to target    | Raw, untransformed data                                          | Cleaned, structured, transformed data                   |
| Processing power         | Leverages power of the target data store                         | Relies on dedicated ETL engine or staging server        |
| Data ingestion speed     | Typically faster to load data initially                          | Can be slower due to upfront transformation processing  |
| Flexibility for new uses | High, as raw data is available for re-transformation             | Lower, as transformations are pre-defined               |
| Schema handling          | Well-suited for schema-on-read                                   | Often relies on schema-on-write                         |
| Data type suitability    | Excellent for structured, semi-structured, and unstructured data | Best for structured, some semi-structured data          |
| Resource utilization     | Optimizes use of scalable cloud data warehouses                  | May require separate infrastructure for transformations |

# ETL vs ELT

Why do companies pay for ELT tools?

Because an engineer's salary is more expensive than software.

By automating the "E" and "L"

engineers focus on Data Governance and Modeling

*the parts that actually provide value to the business.*

ETL

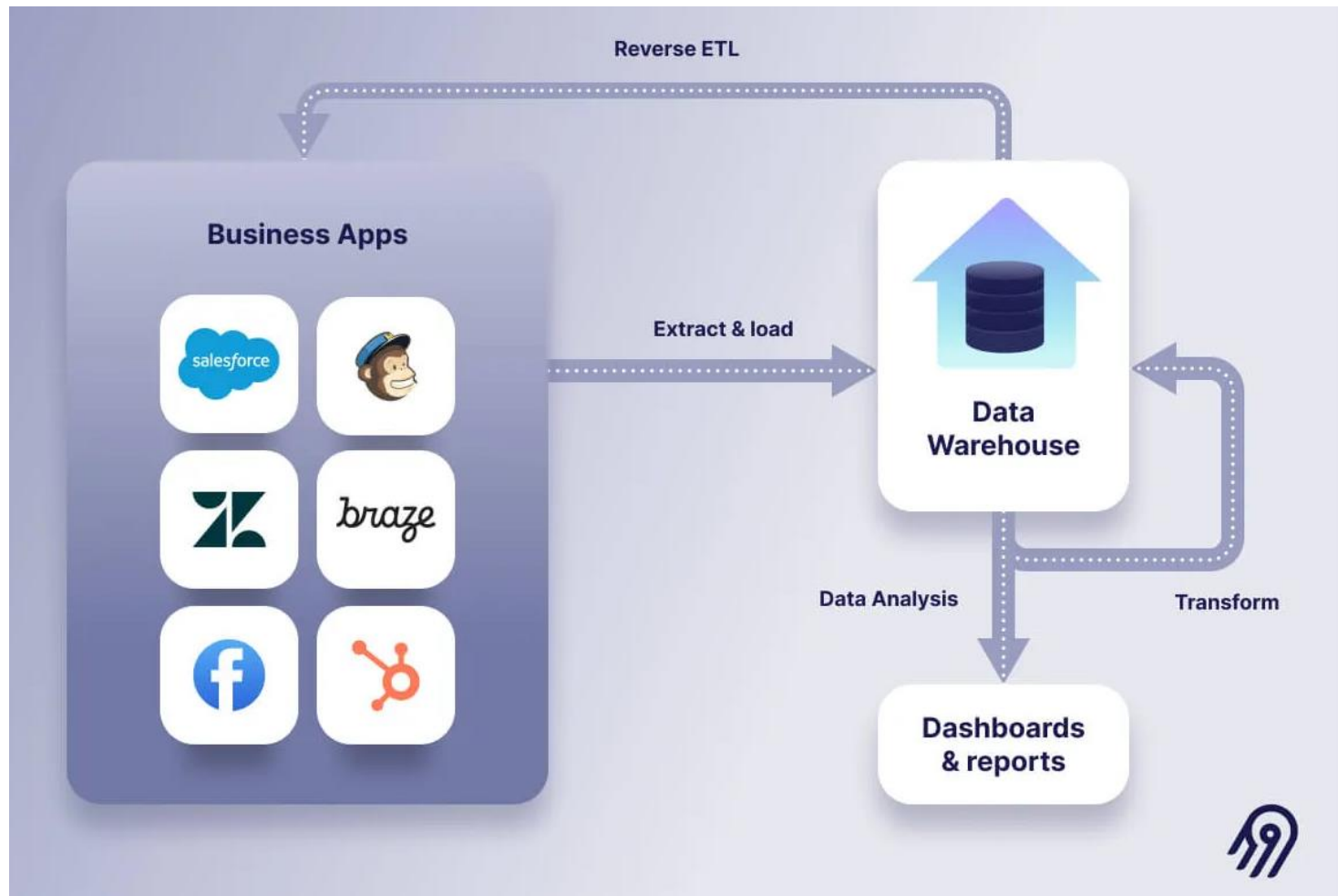
is a "Schema-on-Write" approach

ELT

is a "Schema-on-Read" approach.



# Reverse ETL



# Batch/Stream

## Batch processing

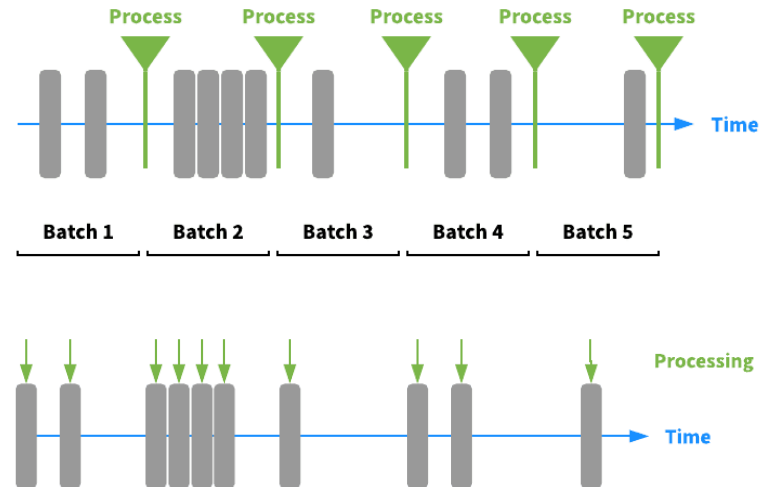
- newly arriving data elements are collected into a group
- group is then processed at a future time (as a batch)
- when each group is processed can be determined
- “microbatch”

## Stream processing

- each new piece of data is processed when it arrives.
- there is no waiting until the next batch processing interval
- data is processed as individual pieces rather than being processed a batch at a time

## ETL with stream processing

- using a modern stream processing framework
- pull data in real time from source, manipulate it on the fly and load it to a target system



# Push/Pull of data

## The Pull Model (Source Polling)

In a pull-based architecture, the target system or the data pipeline orchestration tool (like Airflow, Fintel, or an ETL tool) actively requests data from the source system.

*Typical Use Case - scheduled Batch ETL, API pagination, daily database dumps.*

## The Push Model (Event-Driven)

In a push-based architecture, the source system is responsible for initiating the data transfer.

The moment an event occurs, the source system immediately packages that data and "pushes" it downstream.

*Typical Use Case - Real-time Streaming, Webhooks, Change Data Capture (CDC).*

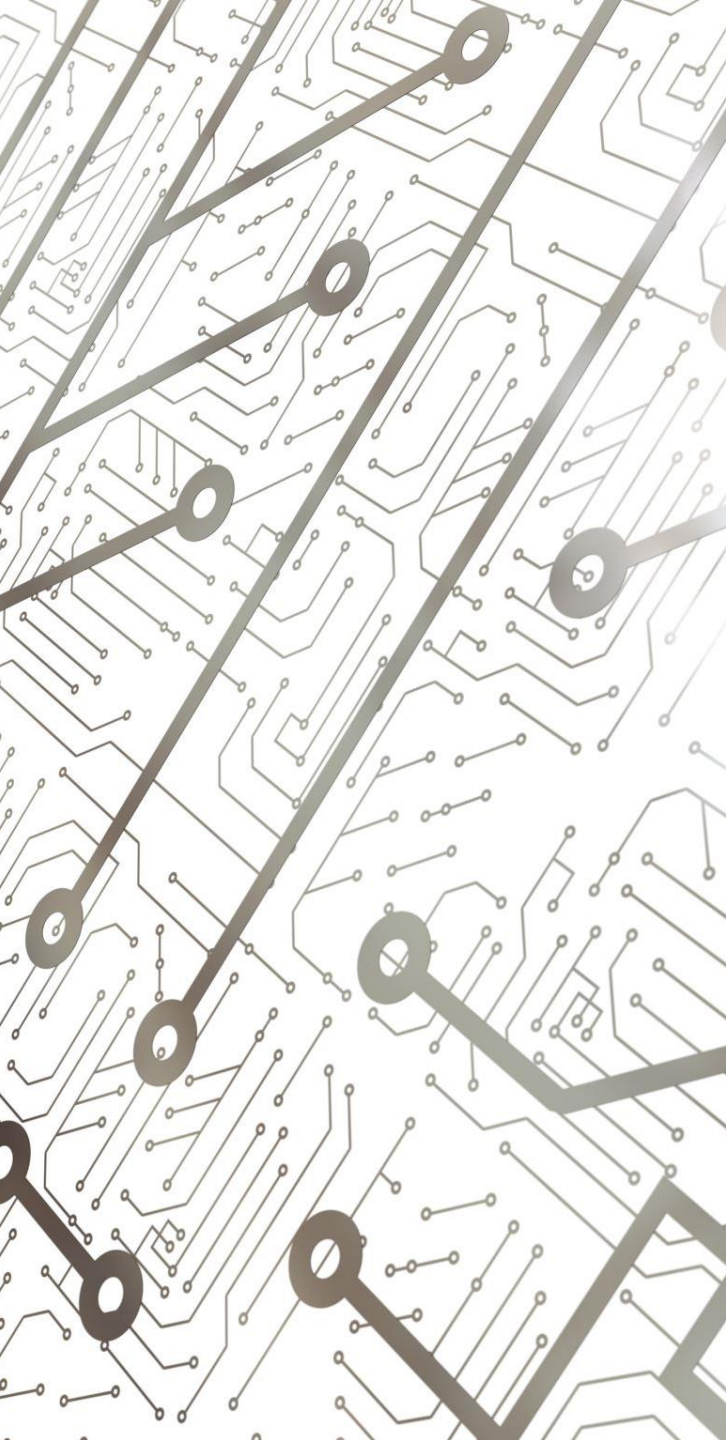
## Pull $\neq$ Batch and Push $\neq$ Stream

modern data architectures often mix them.

For example, a source system might

**Push** real-time event logs into a streaming buffer like Kafka.

a downstream application might then **Pull** micro-batches of data out of Kafka at its own pace to avoid crashing.

A detailed, stylized illustration of a circuit board pattern, featuring a complex network of thin lines and larger circular components, rendered in a light gray tone on the left side of the image.

# ETL System Design

# ETL SYSTEM

## ETL system

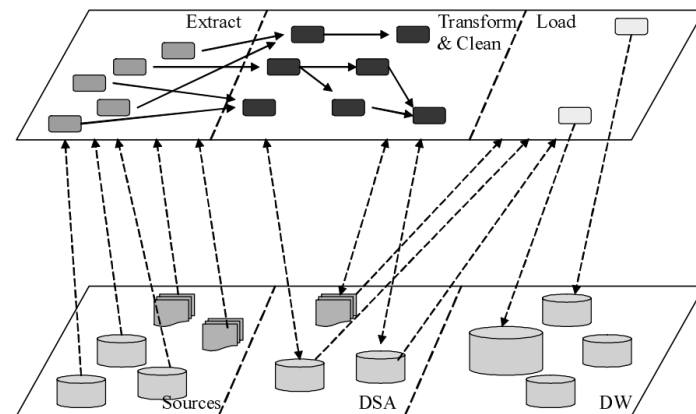
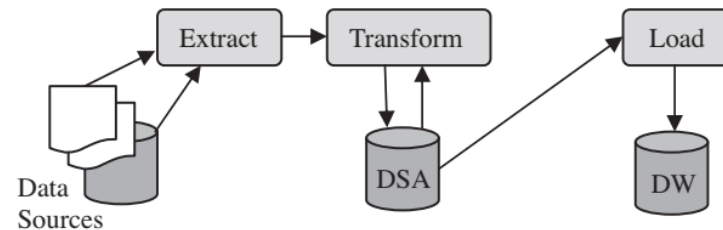
ETL process is a recurring activity and should be automated

ETL requires a storage and processing engine on its own

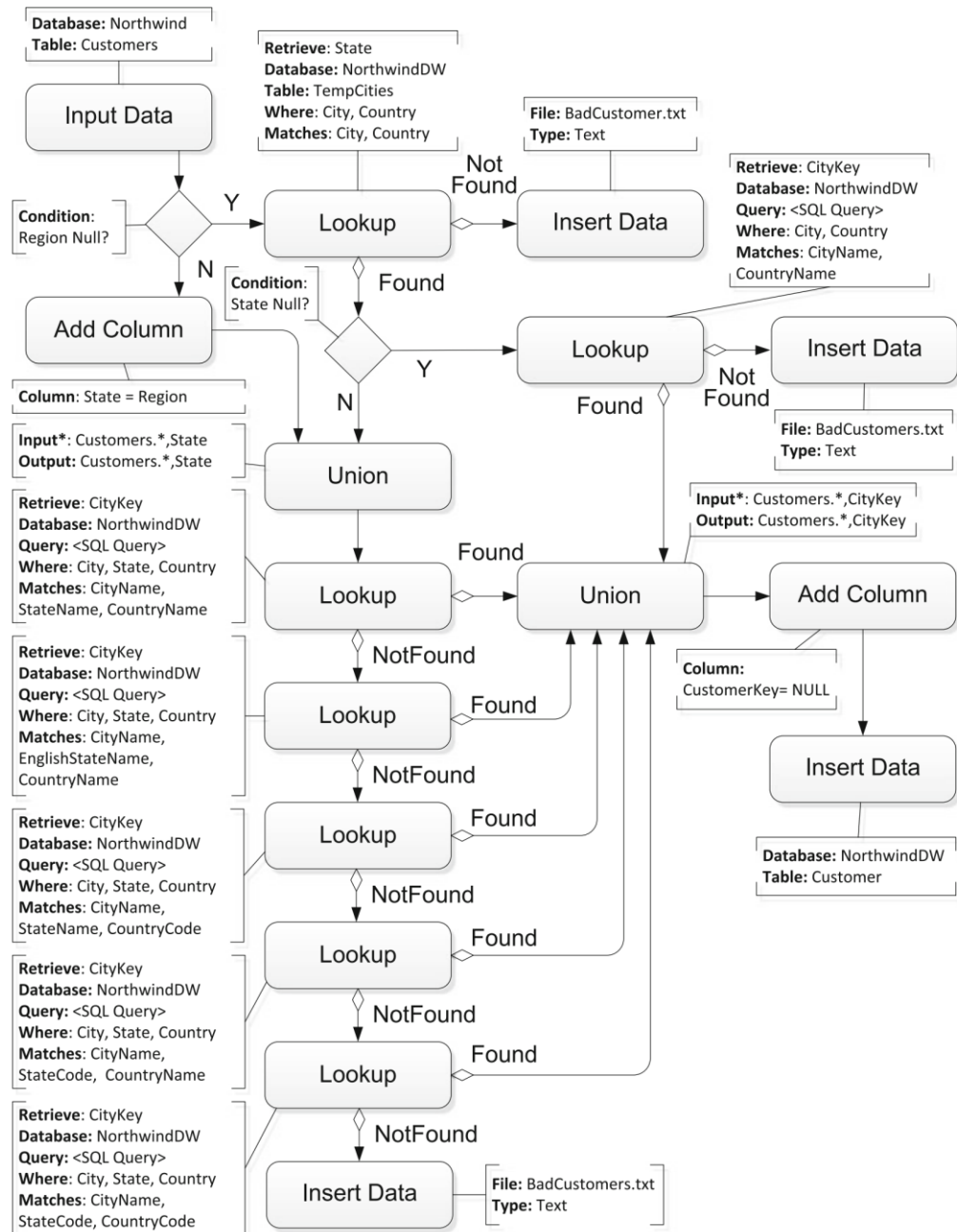
*ETL infrastructure*

*DSA – data staging area*

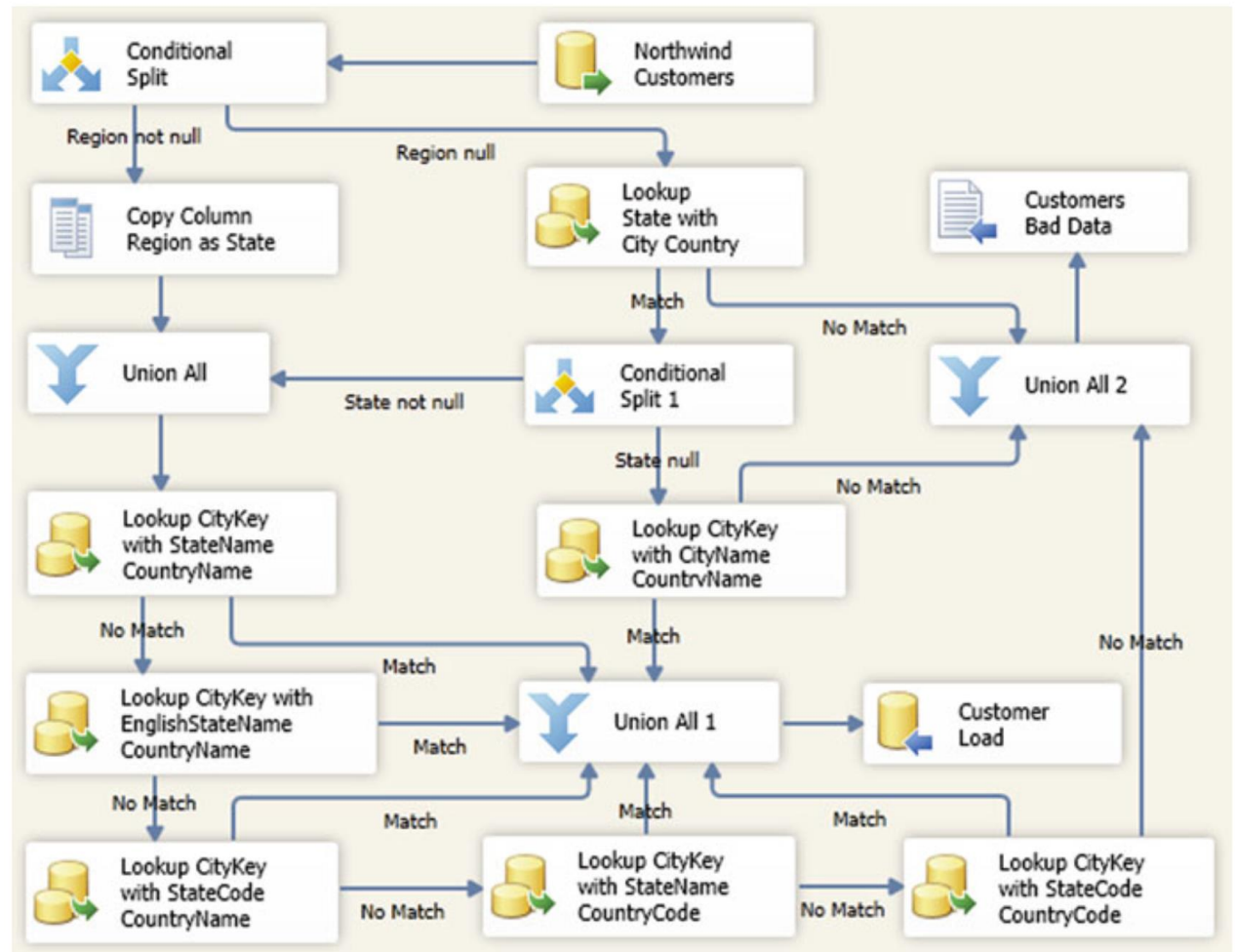
ETL tools support the design, implementation and maintenance of ETL process



# Example Data Flow



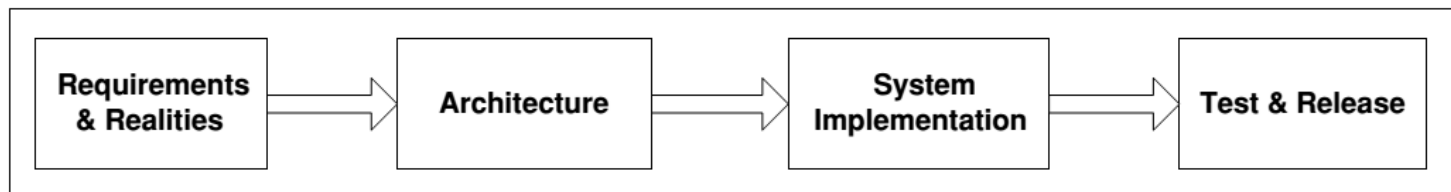
# Example Data Flow



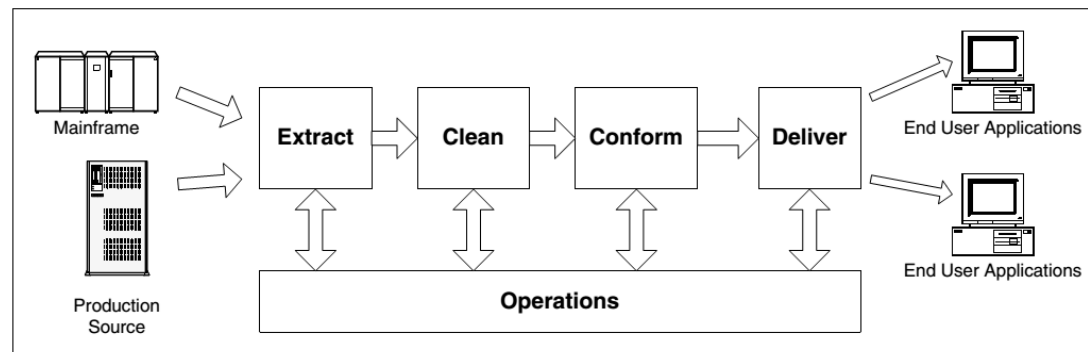
# ETL System

Two simultaneous threads must be kept in mind when building an ETL system:

the **Planning & Design** thread



and the **Data Flow** thread.





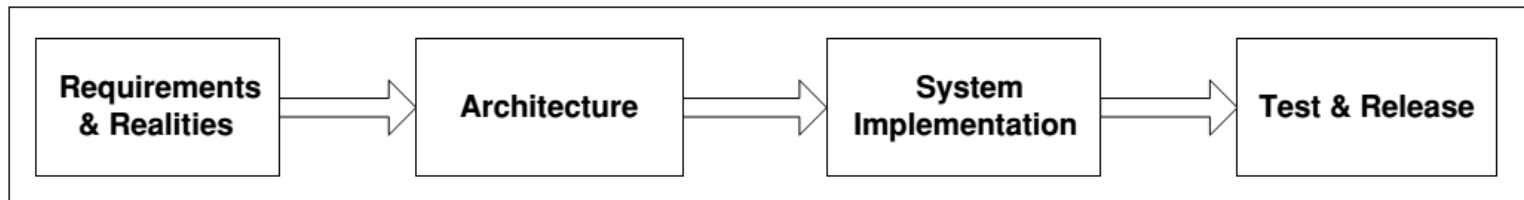
# Planning & Design

## Planning & Design thread

The first step is accounting for all the **requirements and realities**

*These include:*

- Business needs
- Data profiling and other data-source realities
- Compliance requirements
- Security requirements
- Data integration
- Data latency
- Archiving and lineage



# Planning & Design

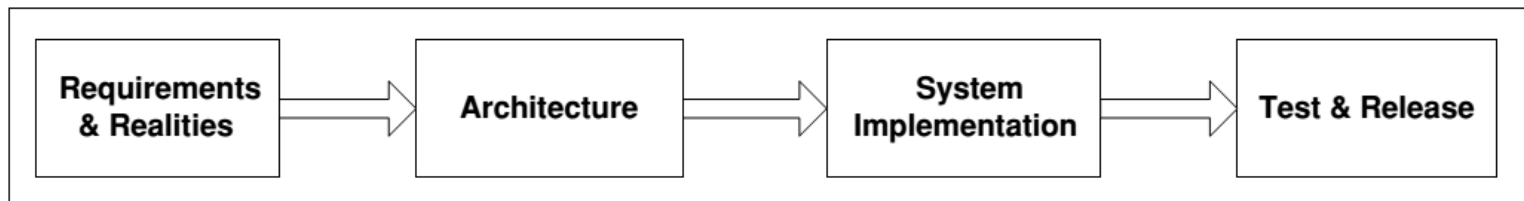
## Planning & Design thread

The second step in this thread is the **architecture** step.

*big decisions about the way we are going to build our ETL system*

*These decisions include:*

- Hand-coded versus ETL vendor tool
- Batch versus streaming data flow
- Horizontal versus vertical task dependency
- Scheduler automation
- Exception handling
- Quality handling
- Recovery and restart
- Metadata
- Security



# Planning & Design

## Planning & Design thread

The third step is system **implementation**.

*spent some quality time on the previous two steps before charging into the implementation!*

This step includes:

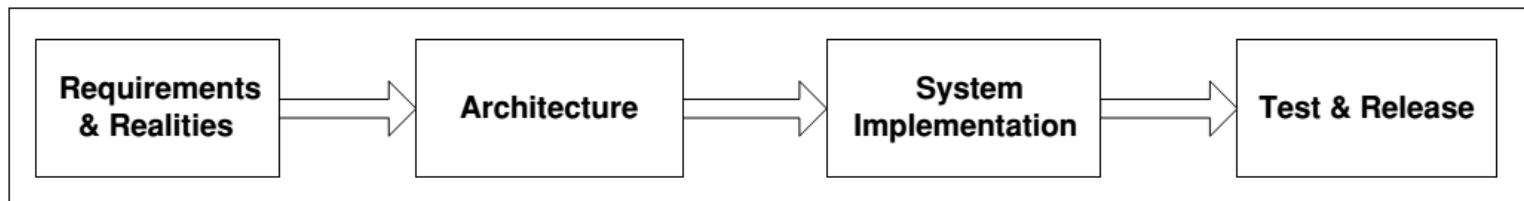
*Hardware*

*Software*

*Coding practices*

*Documentation practices*

*Specific quality checks*



# Planning & Design

## Planning & Design thread

The final step - **test and release**

*is as important as the more tangible designs of the preceding two steps*

Test and release includes the design of the:

*Development system*

*Test systems*

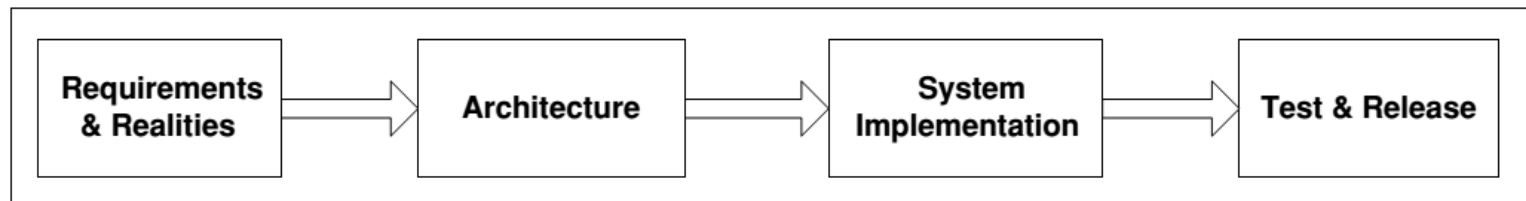
*Production systems*

*Handoff procedures*

*Update propagation approach*

*System snapshoting and rollback procedures*

*Performance tuning*



# ETL system

ETL system has four major components:

Extracting.

*Gathering raw data from the source systems and usually staging the data in the ETL environment before any significant restructuring of the data takes place.*

Cleaning and conforming.

*Sending source data through a series of processing steps in the ETL system to improve the quality of the data received from the source, and merging data from two or more sources to create and enforce conformed dimensions and conformed metrics.*

Delivering.

*Physically structuring and loading the data into the presentation server's target dimensional models.*

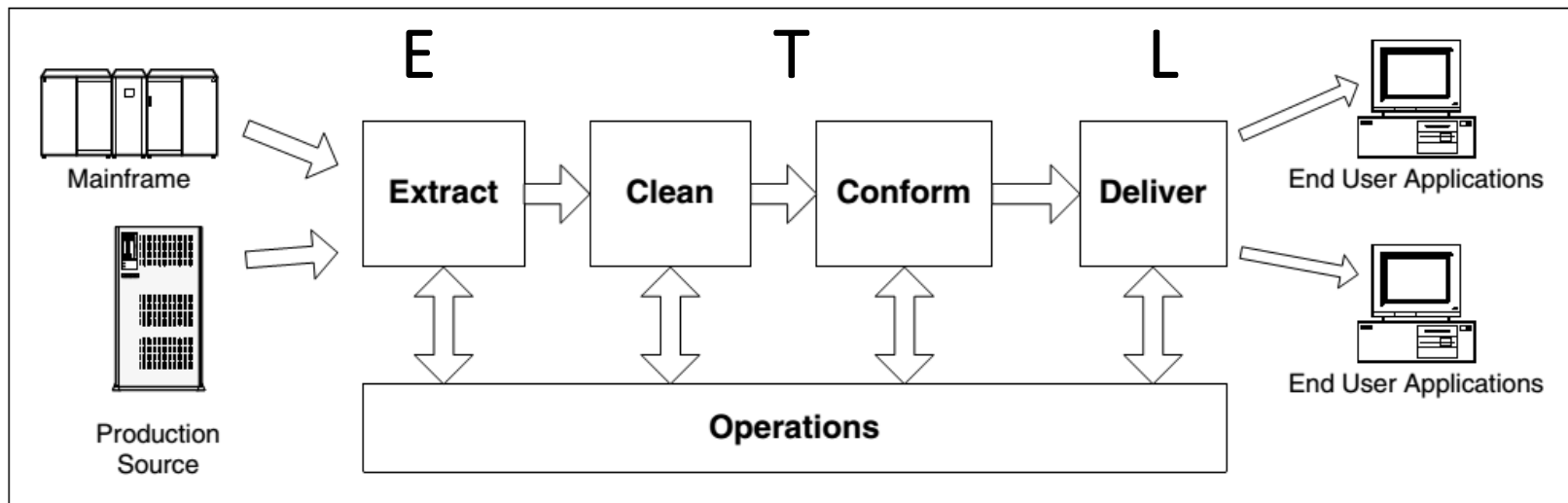
Managing.

*Managing the related systems and processes of the ETL environment in a coherent manner.*

# Data Flow

The Data Flow thread is probably more recognizable  
it is a simple generalization of the E-T-L extract-transform-load scenario

*Extract → clean → conform → deliver*



# Data Flow

## The Data Flow

The extract step includes:

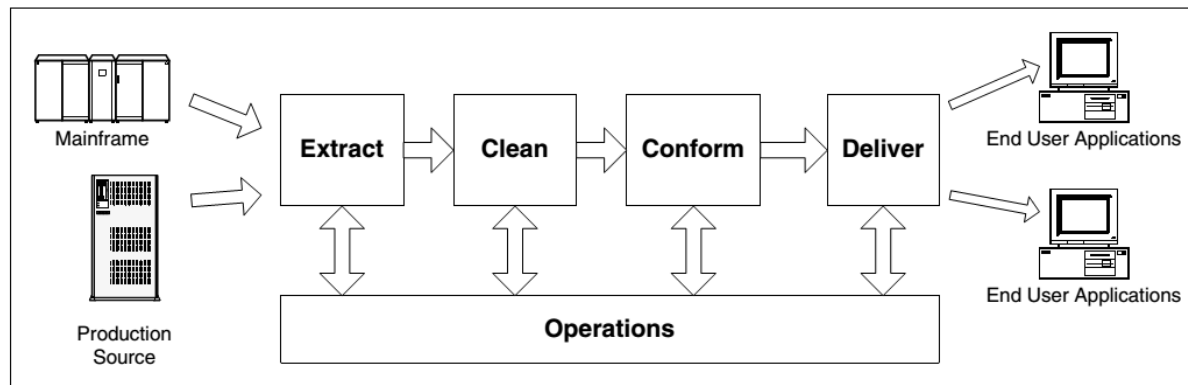
*Reading source-data models*

*Connecting to and accessing data*

*Scheduling the source system, intercepting notifications and daemons*

*Capturing changed data*

*Staging the extracted data to disk*



# Data Flow

## The Data Flow

The clean step involves:

*Enforcing column properties*

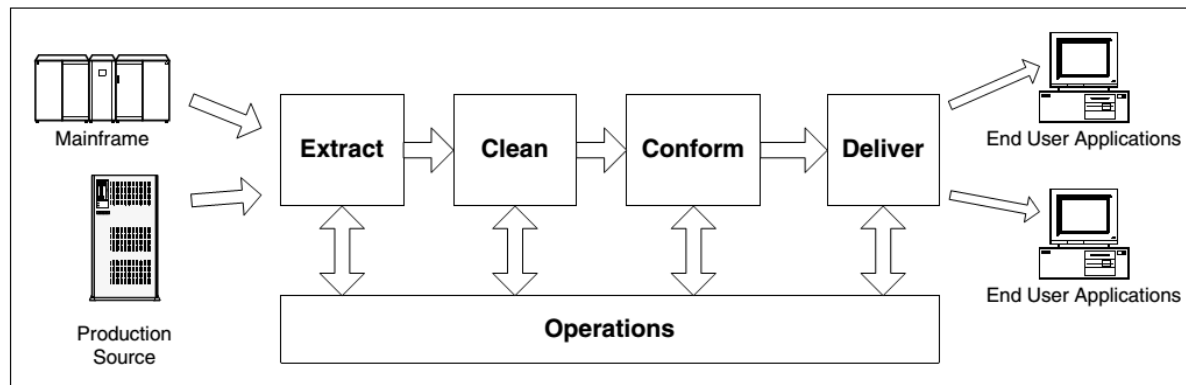
*Enforcing structure*

*Enforcing data and value rules*

*Enforcing complex business rules*

*Building a metadata foundation to describe data quality*

*Staging the cleaned data to disk*





# Data Flow

## The Data Flow

Clean step is followed closely by the conform step, which includes:

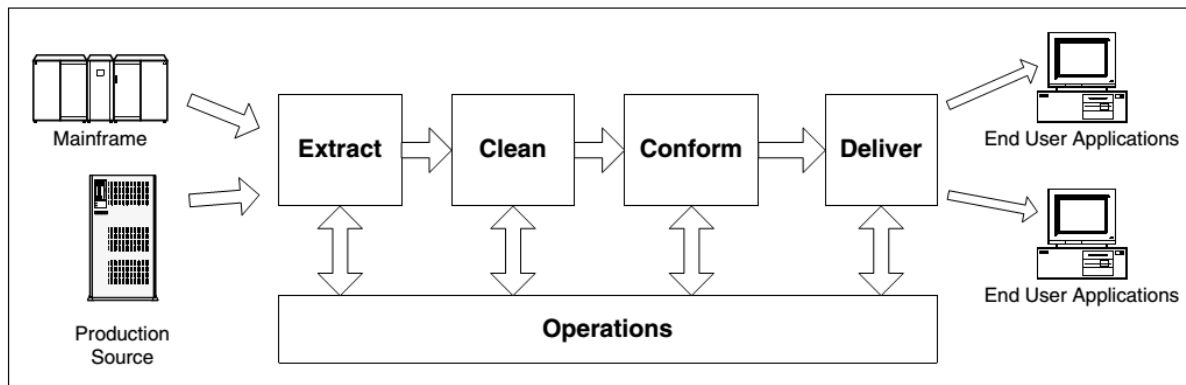
*Conforming business labels (in dimensions)*

*Conforming business metrics and performance indicators (in fact tables)*

*Deduplicating*

*Internationalizing*

*Staging the conformed data to disk*



# Data Flow

## The Data Flow

Finally deliver of data to the end-user application.

Data delivery from the ETL system includes:

*Loading flat and snowflaked dimensions (Loading subdimensions)*

*Generating time dimensions*

*Conforming dimensions and conforming facts*

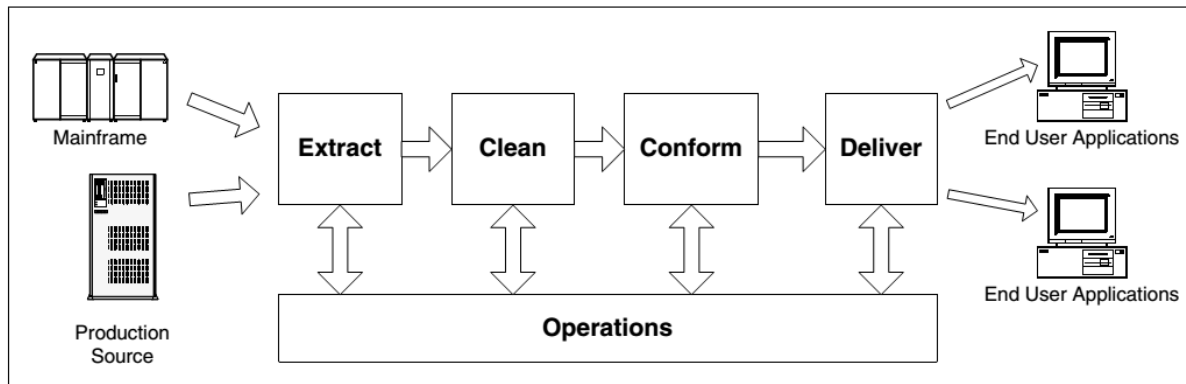
*Loading text facts in dimensions*

*Running the surrogate key pipeline for fact tables*

*Loading three fundamental fact table grains*

*Loading and updating aggregations*

*Staging the delivered data to disk*





Before  
the ETL

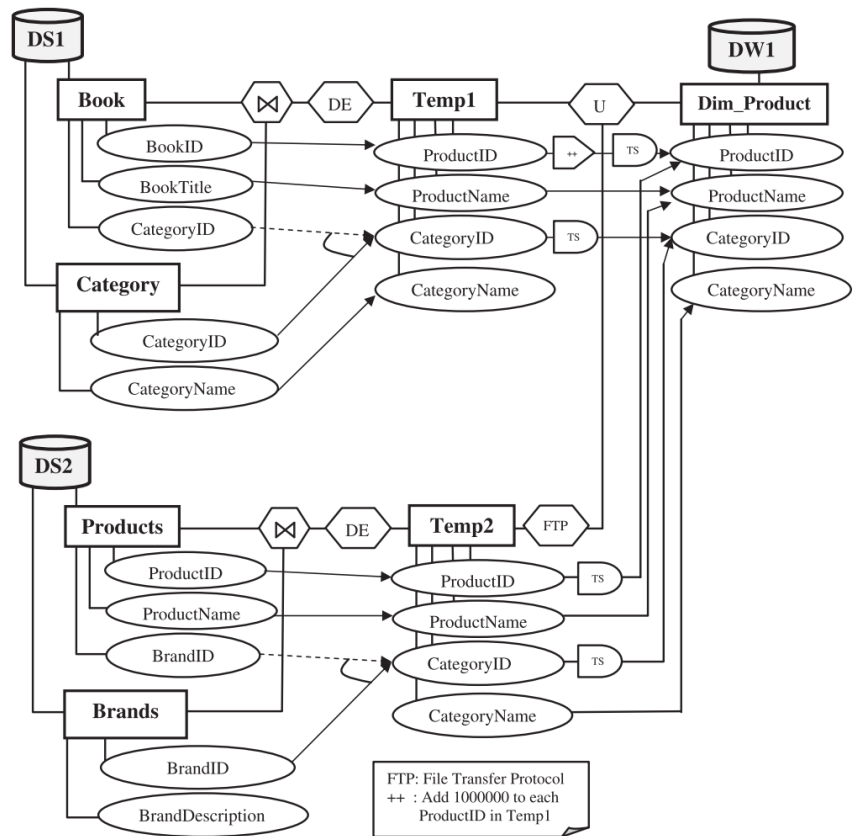
# ETL

## Design

Always Logical Before Physical

Have a plan

*The ETL process must be figured out logically and documented*



**Figure 16** EMD scenario for building products dimension.

# ETL

Design (continued):

Identify data source candidates

*Starting with the highest-level business objectives*

identify the likely candidate data sources you believe will support the decisions needed by the business community

identify specific data elements you believe are central to the end user data

These data elements are then the inputs to the data profiling step

Analyze source systems with a data-profiling tool

*Data in the source systems must be scrutinized for data quality and completeness*

# ETL

## Designing Logical Before Physical (continued):

Receive walk-through of data lineage and business rules

*The data-profiling step should have created two subcategories of ETL-specific business rules:*

- Required alterations to the data during the data-cleaning steps

- Coercions to dimensional attributes and measured numerical facts  
to achieve standard conformance across separate data sources

Receive walk-through of data warehouse data model

*must have a thorough understanding of how dimensions, facts, and other special tables in the dimensional model work together*

Validate calculations and formulas

*Verify with end users any calculations specified in the data lineage  
measure twice, cut once*

# ETL

## Dimensional Data Structures

Dimensional data structures are the target of the ETL processes, and they sit at the boundary between the back room and the front room

In many cases, the dimensional tables will be the final physical-staging step before transferring the tables to the end user environments

*Fact tables*

*Dimension tables*

*Surrogate key mapping tables*

# ETL

## Logical Design First

1. Have a (preliminary) plan
2. Identify data source candidates
3. Analyze source systems with a data-profiling tool

*Possible STOP here*

4. Receive walk-through of data lineage and business rules

*Required alterations to the data during the data-cleaning steps*

*Coercions to dimensional attributes and measured numerical facts to achieve standard conformance across separate data sources*

5. Receive walk-through of data warehouse data model
6. Validate calculations and formulas (*measure twice, cut once*)



# Staging

## Stage

typically transformed data is not directly loaded into the target data warehouse

this data should first enter a staging database

*making it easier to roll back if something goes wrong*

at this stage it is easier to

*generate audit reports for regulatory compliance*

*diagnose and repair data problems*

# Staging

The back room area of the data warehouse has frequently been called the staging area.

Staging in this context means writing to disk

The staging area stores data on its way to the final presentation area of the data warehouse.

It is recommended to stage data at the four major checkpoints of the ETL data flow:

Extract → Clean → Conform → Deliver

To stage your data or not depends on two conflicting objectives:

Getting the data from the originating source to the ultimate target as fast as possible

Having the ability to recover from failure without restarting from the beginning of the process

# Staging

When the staging area is initially set up  
the ETL architect must supply an overall data storage  
measure of the staging area  
*estimate the space allocations*  
*and parameter settings for the staging database, file systems, and*  
*directory structures*  
staging area volumetric worksheet  
*focusing on the final delivery tables at the end of the ETL data flow*

## Heterogeneous Staging Area

Heterogeneous source systems may call for a heterogeneous  
data staging area

# Staging

## Staging area volumetric worksheet

| Table Name            | Update Strategy   | Load Frequency | ETL Job(s)         | Initial Rowcount | Avg Row Length | Grows with                          | Expected Monthly Rows | Expected Monthly Bytes | Initial Table Size Bytes | Table Size 6 mo. (MB) |
|-----------------------|-------------------|----------------|--------------------|------------------|----------------|-------------------------------------|-----------------------|------------------------|--------------------------|-----------------------|
| S_ACCOUNT             | Truncate / Reload | Daily          | SAccount           | 39,933           | 27             | New accounts                        | 9,983                 | 269,548                | 1,078,191                | 2.57                  |
| S_ASSETS              | Insert / Delete   | Daily          | SAssets            | 771,500          | 78             | New assets                          | 192,875               | 15,044,250             | 60,177,000               | 143.47                |
| S_BUDGET              | Truncate / Reload | Monthly        | SBudget            | 39,932           | 104            | Refreshed monthly                   | 9,983                 | 1,038,232              | 4,152,928                | 9.90                  |
| S_COMPONENT           | Truncate / Reload | On demand      | SComponent         | 21               | 31             | Components added to Inventory       | 5                     | 163                    | 651                      | 0.00                  |
| S_CUSTOMER            | Truncate / Reload | Daily          | SCustomer          | 38,103           | 142            | New customers added daily           | 9,526                 | 1,352,657              | 5,410,626                | 12.90                 |
| S_CUSTOMER_HISTORY    | Truncate / Reload | Daily          | SCustomerHistory   | 2,307,707        | 162            | Refresh with each bulk load         | 576,927               | 93,462,134             | 373,848,534              | 891.32                |
| S_CUSTOMER_TYPE       | Truncate / Reload | On demand      | SCustomerType      | 5                | 21             | New customer types                  | 1                     | 26                     | 105                      | 0.00                  |
| S_DEFECT              | Truncate / Reload | On demand      | SDefect            | 84               | 27             | New defect names                    | 21                    | 567                    | 2,268                    | 0.01                  |
| S_DEFECTS             | Insert Only       | Daily          | SDefects           | 8,181,132        | 132            | Transaction defects                 | 2,045,283             | 269,977,356            | 1,079,909,424            | 2,574.70              |
| S_DEPARTMENT          | Truncate / Reload | On demand      | SDepartment        | 45               | 36             | Departments established             | 11                    | 405                    | 1,620                    | 0.00                  |
| S_FACILITY            | Truncate / Reload | Daily          | SFacility          | 45,260           | 32             | New or changed facilities worldwide | 11,315                | 362,080                | 1,448,320                | 3.45                  |
| S_HISTORY_FAIL_REASON | Insert / Delete   | On demand      | SHistoryFailReason | 6                | 27             | New failure codes                   | 2                     | 41                     | 162                      | 0.00                  |
| S_OFFICE              | Truncate / Reload | Daily          | SOffice            | 14               | 56             | New offices opened                  | 4                     | 196                    | 784                      | 0.00                  |
| S_PACKAGE_MATL        | Truncate / Reload | Daily          | SPackageMatl       | 54               | 18             | New packaged material categories    | 14                    | 243                    | 972                      | 0.00                  |
| S_PRODUCT             | Truncate / Reload | Daily          | SProduct           | 174,641          | 73             | New products                        | 43,660                | 3,187,198              | 12,748,793               | 30.40                 |
| S_PROVIDER            | Truncate / Reload | On demand      | SProvider          | 63               | 45             | Service providers                   | 16                    | 709                    | 2,835                    | 0.01                  |
| S_REGION              | Truncate / Reload | Daily          | SRegion            | 333              | 37             | New or changed global regions       | 83                    | 3,080                  | 12,321                   | 0.03                  |
| S_RESPONSES           | Insert Only       | Daily          | SResponses         | 5,199,095        | 105            | Response transaction                | 1,299,774             | 136,476,244            | 545,904,975              | 1,301.54              |
| S_SURVEY              | Truncate / Reload | Daily          | SSurvey            | 45,891           | 83             | Survey conducted                    | 11,473                | 952,238                | 3,808,953                | 9.08                  |

# Staging

## Integrity Checking

It is a good practice to have integrity checks in the ETL process rather than in the staging database

The ETL process must know how to handle data anomalies in a more automatic way – it cannot simply reject all data

*business rules for different data-quality scenarios*

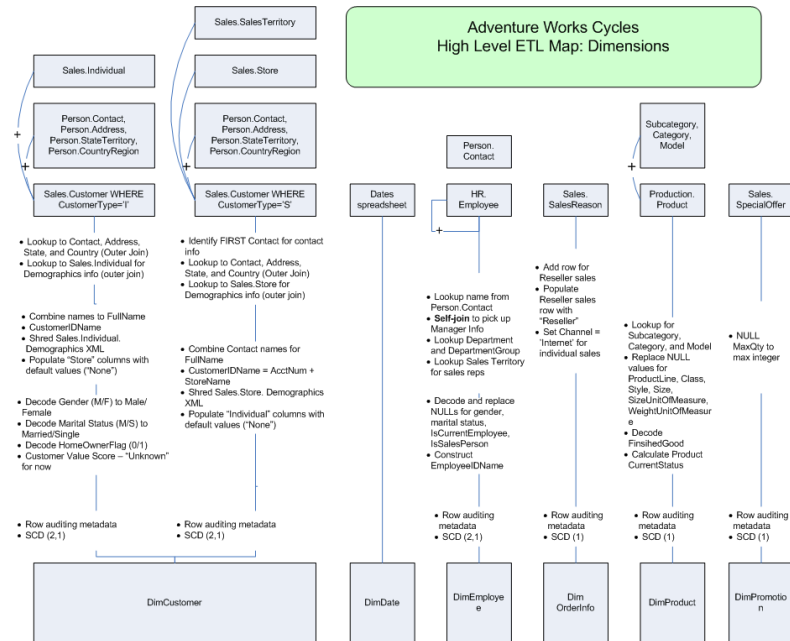
If the data is unacceptable, it has to be rejected completely and put it into a reject file for investigation.

*Semi-automatic*

# ETL

ETL development starts out with  
Preparing the logical map and high-level plan  
*independent of any specific technology or approach*

| Target DW         |                    |           |            |     | Data Sources  |             |                 | Transformation |
|-------------------|--------------------|-----------|------------|-----|---------------|-------------|-----------------|----------------|
| Table Name        | Column Name        | Data Type | Table Type | SCD | Database Name | Table Name  | Column Name     | Data Type      |
| tf101masukpelajar | ff101kat_pelajar   | Number    | Fact       | n/a | ASIS          | t210student | f210kat_pelajar | Char(4)        |
| tf101masukpelajar | ff101kdprogram     | Number    | Fact       | n/a | ASIS          | t210student | f210kdprogram   | Char(4)        |
| tf101masukpelajar | ff101jantina       | Number    | Fact       | n/a | ASIS          | t210student | f210jantina     | Char(1)        |
| tf101masukpelajar | ff101bangsa        | Number    | Fact       | n/a | ASIS          | t210student | f210bangsa      | Char(1)        |
| ff101masukpelajar | ff101student_total | Number    | Fact       | n/a | ASIS          | T210student |                 | Char(4)        |
| td008jantina      | fd008kdjantina     | Char(1)   | Dimension  | 2   | ASIS          | t012jantina | f012kdjantina   | Char(1)        |
| td008jantina      | fd008keterangan    | Char(15)  | Dimension  | 2   | ASIS          | t012jantina | f012ketarangan  | Char(15)       |
| td006bangsa       | fd006kdbangsa      | Char(1)   | Dimension  | 2   | ASIS          | t013bangsa  | f013kdbangsa    | Char(1)        |



# ETL

All the dimension tables must be processed before the key lookup steps for the fact table.

The dimension tables are usually independent from each other, but sometimes they also have processing dependencies.

*It's important to clarify these dependencies, as they become fixed points around which the job control flows.*

ETL specification should describe

- historic load strategies for each target table

- incremental load strategies for each target table

*incremental processing, by contrast, must be fully automated*

Occasionally, the same ETL code can perform both the initial historic load and ongoing incremental loads

- more often you build separate ETL processes

# ETL

## Historic load

### Start with dimensions

*Start with permanent dimensions and user defined ones*

*Then go for type-1*

*Then go for other dims*

*Remember about*

- Doing transformations – like handling nulls, decoding production codes

- Conform data

- Handle many-to-many relations

- Apply surrogate keys

- Load efficiency

### Next go to fact tables

*Remember about*

- Handling null values

- Improving fact table – calculated measures

- Pipelining the surrogate key lookup

- Load efficiency



# ETL

The steps to loading into a partitioned table include:

1. Disable foreign key (referential integrity) constraints between the fact table and each dimension table before loading data.
2. Drop or disable indexes on the fact table.
3. Load the data using fast-loading techniques.
4. Create or enable fact table indexes.
5. If necessary, perform steps to stitch together the table's partitions.
6. Confirm each dimension table has a unique index on the surrogate key column.
7. Enable foreign key constraints between the fact table and dimension tables.

# ETL

Incremental load

Highly automated

*Scheduling, Exception and error handling (automated for predicted, graceful for unpredicted), logging and audit*

Begin with the dimension tables

*Identify new and changed dimension rows*

Apply proper mechanisms – depending on the SCD type

Remember about doing transformations, conforming data, handling many-to-many relations, apply surrogate keys

Next go for fact tables

*Identify new and changed fact rows*

Apply proper surrogate key pipelines – problem with late arriving facts

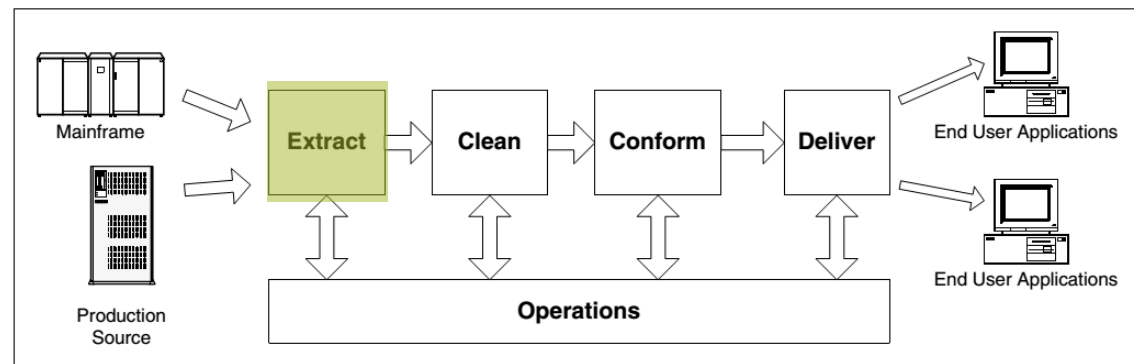
Problem with aggregates

Problem with real-time delivery

# The extract step includes:

- Reading source-data models
- Connecting to and accessing data
- Capturing changed data
- Staging the extracted data to disk

# Extract



# Extraction

## Extract

focal point is on

*how to interface to the required  
source systems for your project*

*how to examine the data sources -  
analysis*

not only are the systems  
separated and acquired at  
different times

*but frequently they are logically  
and physically incompatible*

once you understand what the  
target needs to look like

*you need to identify and examine  
the data sources*

## Traditionally

extraction meant getting data  
from Excel files and RMDS

## With the increase in Software as a Service (SaaS) applications

the majority of businesses now  
find valuable information in the  
apps themselves,

## Today,

data extraction is mostly about  
obtaining information from an  
app's storage via APIs or  
webhooks.

# Extraction

## Steps in data extraction

### Initial extraction

*First time data extraction*

### Ongoing extraction

*Just new data*

*Changed data or even deleted data*

## Initial load

capturing source data content changes is not important because you load all data from a point in time forward

*Full refresh*

## Ongoing load

many data warehouse tables are so large that they cannot be refreshed during every ETL cycle

*Need to transfer only the relevant changes to the source data since the last update*

Incremental update

*Capturing data changes is far from a trivial task.*

# Extraction

Extract architecture design

3 approaches:

*Full-extraction.*

Each extraction collects all data from the source and pushes it down the data pipeline.

*Incremental extraction.*

At each new cycle of the extraction process (e.g. every time the ETL pipeline is run), only the new data is collected from the source, along with any data that has changed since the last collection.

High-Water Mark – we look at the target warehouse's max *updated\_at* date and only query records in the source that are greater than that value.

cannot detect "Deletes" – If a record is deleted in the source, it stays forever in the target.

*Source-driven extraction.*

The source notifies the ETL system that data has changed, and the ETL pipeline is run to extract the changed data.

# Extraction

## Data Extraction:

Too time consuming to ETL all data at each load

*Can take days/weeks*

*Drain on the operational systems and DW systems*

Extract/ETL only changes since last load

*Delta = changes since last load*

## Data Extraction:

Common techniques are used to limit the amount of operational data scanned at the point of refreshing the data warehouse:

*DB triggers*

*Change Data Capture*

*Partitioning*

*Before / After image*

*Audit cols / Timestamp*

*Delta file*

*Log file scrapping*

*Message queue monitoring*

# Extraction

## Data Extraction:

Common techniques are used to limit the amount of operational data scanned at the point of refreshing the data warehouse:

### *DB triggers*

Triggers for INSERT, UPDATE, and DELETE on a single table  
triggers write information about the record change to 'change tables'

### *Change Data Capture*

records INSERTs, UPDATEs, and DELETEs applied to tables

Every database has a "Write-Ahead Log" (WAL) or "Binlog." Every single change (Insert, Update, Delete) is written there before the data is actually changed in the table.

CDC tools (like Debezium) "tail" this log.

makes a record of what changed, where, and when, in a simple relational 'change tables'



# Extraction

## Data Extraction:

Common techniques are used to limit the amount of operational data scanned at the point of refreshing the data warehouse:

### *Partitioning:*

Some source systems might use range partitioning

the source tables are partitioned along a date key, which allows for easy identification of new data

For example, if you are extracting from an orders table, and the orders table is partitioned by week, then it is easy to identify the current week's data

### *Before / After image*

a “before” and an “after” image of the operational file together compared to each other to determine the activity that has transpired

# Extraction

## Data Extraction:

Common techniques are used to limit the amount of operational data scanned at the point of refreshing the data warehouse:

### *Timestamp*

Scan data that has been timestamped in the operational environment

### *Delta file*

limiting the data to be scanned is to scan a “delta” file

### *Log file*

scan a log file or an audit file created as a by-product of transaction processing.

# Extraction

## Timestamps

Additional columns in the source system

*audit columns appended to the end of each table to store the date and time a record was added or modified*

*audit columns should be populated via database triggers fired off automatically (instead of front-end applications)*

## Idea

*Extract all source records for which the last audit (created or modified) date and time is greater than the maximum audit date and time from the last load*

There should be a special ETL last-change table that for each source table captures the maximum date time found in the source system audit columns at the time of each extract

# Extraction

Timestamped

| ID | CustomerID | ProductDescription | Qty | Revenue | Sales Date  |
|----|------------|--------------------|-----|---------|-------------|
| 1  | 1          | White sheet (A4)   | 100 | 4.00    | 22-Mar-2012 |
| 2  | 1          | James Clip (Box)   | 1   | 2.50    | 22-Mar-2012 |
| 3  | 2          | Whiteboard Marker  | 1   | 2.00    | 22-Mar-2012 |
| 4  | 3          | Letter Envelop     | 200 | 75.00   | 23-Mar-2012 |
| 5  | 1          | Paper Clip         | 12  | 4.00    | 23-Mar-2012 |

Batch

| CustomerID | CustomerName | Type       | Entry Date  |
|------------|--------------|------------|-------------|
| 1          | John         | Individual | 22-Mar-2012 |
| 2          | Ryan         | Individual | 22-Mar-2012 |
| 3          | Bakers'      | Corporate  | 23-Mar-2012 |

| Batch_ID | Loaded_Until | Status  |
|----------|--------------|---------|
| 1        | 22-Mar-2012  | Success |
| 2        | 23-Mar-2012  | Success |

```
SELECT t.*  
FROM Customer t  
WHERE t.entry_date > (select nvl(  
max(b.loaded_until),  
to_date('01-01-1900', 'MM-DD-YYYY')  
)  
from batch b  
where b.status = 'Success');
```

The diagram illustrates the extraction process. A green arrow points from the 'Loaded\_Until' column of the 'Batch' table to the 'WHERE' clause of the SQL query, specifically to the 'max(b.loaded\_until)' function. Another green arrow points from the 'CustomerID' column of the 'Batch' table to the 'FROM Customer t' part of the query. A third green arrow points from the 'Status' column of the 'Batch' table to the 'where b.status = 'Success'' condition of the query.

# Extraction

## Timestamped

| ID | CustomerID | ProductDescription | Qty | Revenue | Sales Date  |
|----|------------|--------------------|-----|---------|-------------|
| 1  | 1          | White sheet (A4)   | 100 | 4.00    | 22-Mar-2012 |
| 2  | 1          | James Clip (Box)   | 1   | 2.50    | 22-Mar-2012 |
| 3  | 2          | Whiteboard Marker  | 1   | 2.00    | 22-Mar-2012 |
| 4  | 3          | Letter Envelop     | 200 | 75.00   | 23-Mar-2012 |
| 5  | 1          | Paper Clip         | 12  | 4.00    | 23-Mar-2012 |

## Batch

| CustomerID | CustomerName | Type       | Entry Date  |
|------------|--------------|------------|-------------|
| 1          | John         | Individual | 22-Mar-2012 |
| 2          | Ryan         | Individual | 22-Mar-2012 |
| 3          | Bakers'      | Corporate  | 23-Mar-2012 |

| Batch_ID | Loaded_Until | Status  |
|----------|--------------|---------|
| 1        | 22-Mar-2012  | Success |
| 2        | 23-Mar-2012  | Success |

```
SELECT t.*  
FROM Customer t  
WHERE t.entry_date > (select nvl(  
max(b.loaded_until),  
to_date('01-01-1900', 'MM-DD-YYYY')  
)  
from batch b  
where b.status = 'Success');
```

The diagram illustrates the data flow for the SQL query. A green arrow originates from the 'Loaded\_Until' column of the 'Batch' table (specifically pointing to the value '23-Mar-2012' in row 2) and points to the 'nvl' function in the SQL query. Another green arrow points from the 'WHERE' clause of the SQL query to the 'Customer' table, indicating that the query filters customers based on the batch loading information.

# Extraction

## Types of Data Extraction

### Online extraction

#### *Direct*

Source writes data into target or target reads data from source

Security concerns

High coupling / dependencies

### Offline Extraction

#### *Through a transport medium – typically files*

Transfer data by scp, rfts (reliable file transfer system), ESB (enterprise service bus), SOA (service oriented architecture), etc

Often high amounts of data, therefore bulk transfer of compressed data most widely used

Better decoupling of source and target

# Extraction

## Extraction intervals

Depends on the requirements on timeliness of the data warehouse data

Depends on the time needed to do the ETL

## Two major approaches

*Periodically – in regular intervals*

Every day, week, etc.

During times with low usage

*Instantly / Continuous*

Every change is directly propagated into the data warehouse

„real time data warehouse“

## Triggered by a specific request

*Addition of a new product*

## Triggered by specific events

*Number of changes in operational data exceeds threshold*

# Ingestion Patterns

Full Refresh (Truncate and Load):

DROP TABLE or TRUNCATE TABLE followed by a bulk INSERT

Append-Only (Immutable Ingestion)

This pattern assumes that data is never updated or deleted—only new records are added.

New records are simply added to the end of the table. Usually includes an `ingested_at` timestamp.

Upsert (Merge)

This pattern checks if a record already exists based on a primary key. If it does, the record is updated; if not, a new row is created.

SQL MERGE statement

*It matches incoming data on a Primary Key (e.g., `user_id`). If it exists, update; if not, insert.*

SCD (Slowly Changing Dimensions):

If a customer moves from NY to CA, do you overwrite NY (SCD Type 1) or keep both with timestamps (SCD Type 2)?

*ELT usually implements SCD in the Transformation phase.*

Partitioned Ingestion

For datasets in the terabyte or petabyte range, we use Partitioning. Instead of one giant table, data is stored in "folders" or partitions based on a key (usually a date).

You only "Overwrite" or "Append" to a specific partition



# \*Extraction in ETL

The analysis of the source system is usually broken into two major phases:

The data discovery phase

*Understanding the content of the data is crucial for determining the best approach for retrieval*

The analysis of the source system is usually broken into two major phases:

The anomaly detection phase

*Detect abnormal values in the source data*

*Data anomaly is a piece of data which doesn't fit into the domain of the rest of the data it is stored with*

\*Note that in ELT this is typically done in the T step.

# Extraction in ETL

## Data Discovery Phase

Collecting and Documenting Source Systems

*Typical organizations have countless distinct systems*

*Keeping Track of the Source Systems*

Determining the System-of-Record

*system-of-record is the originating source of data*

*most enterprises data is stored redundantly across many different systems*

*the same piece of data is often copied, moved, manipulated, transformed, altered, cleansed, etc.*

## Data Discovery Phase

Significant characteristics that you want to discover during this phase:

*Unique identifiers and natural keys*

*natural key is what the business uses to uniquely describe the row*

*Data types*

*declared vs. actually used*

*Relationships between tables*

*Discrete relationship*

*single look-up table that stores all of the static reference data for all of the tables throughout the database*

*Cardinality of relationships and columns*

*Knowing the cardinality of relationships is necessary to predict the result of your queries*

# Extraction in ETL

## Anomaly Detection Phase

### Data anomaly

*is a piece of data that does not fit into the domain of the rest of the data it is stored with*

### Detecting anomalies

*requires specific techniques and entails analytical scrutiny*

*plan what to expect*

### Why important

*Exposure of unspecified data anomalies once the ETL process has been created is the leading cause of ETL deployment delays*

It is useful to divide the various kinds of data-quality checks into categories

## Types of Enforcement:

Column property enforcement

Structure enforcement

Data and value enforcement

# Extraction in ETL

After the basic strategic assessment is made

- lengthy tactical data profiling effort should occur to squeeze out as many problems as possible

Usually, this task begins during the data modeling process and extends into the ETL system design process.

- Sometimes, the ETL team is expected to include a source with content that hasn't been thoroughly evaluated.

Issues that show up result in detailed specifications that are either sent back to the originator of the data source as requests for improvement

- form requirements for the data quality processing described in cleaning and conforming subsystems

# Extraction in ETL

## Column Property Enforcement

Column property enforcement ensures that incoming data contains expected values from the providing system's perspective.

Useful column property enforcement checks include screens for:

*Null values in required columns*

*Numeric values that fall outside of expected high and low ranges*

*Columns whose lengths are unexpectedly short or long*

*Columns that contain values outside of discrete valid value sets*

*Adherence to a required pattern or member of a set of patterns*

*Hits against a list of known wrong values where list of acceptable values is too long*

*Spell-checker rejects*

# Extraction in ETL

## Structure Enforcement

Focuses on the relationship of columns to each other.

We enforce structure by making sure that tables have proper primary and foreign keys and obey referential integrity.

We check explicit and implicit hierarchies and relationships among groups of fields that

*for example, constitute a valid postal mailing address.*

Structure enforcement also checks hierarchical parent-child relationships to make sure that every child has a parent or is the supreme parent in a family.

# Extraction in ETL

## Data and Value Rule Enforcement

Data and value rules range from simple business rules

*such as if customer has preferred status, the overdraft limit is at least \$1000*

to more complex logical checks

*such as a commercial customer cannot simultaneously be a limited partnership and a type C corporation.*

## Value rules

*an extension of these checks on data and can take the form of aggregate value business rules*

Ex. People in this region are reporting a statistically improbable number of accidents

*Value rules can also provide a probabilistic warning that the data may be incorrect*

Ex. Boy's named Sam

# Extraction in ETL

The diagram illustrates various data quality issues in a table. Callouts point to specific rows or columns:

- PK / Unique Key violated:** Points to the first two rows, which both have a CustomerNo of 1.
- Data not uniform:** Points to the Name column, where the first row has a comma in the name.
- Not valid:** Points to the Age column, where the value 15 is present for two different birthdates.
- Unknown / missing:** Points to the Address code column, where the first row has a NULL value.
- Inconsistent:** Points to the Age column, where the value 22 is present for two different birthdates.
- Wrong value:** Points to the Gender column, where the value 'Q' is present.
- FK violated:** Points to the Address code column, where the value 7589 is present.

| CustomerNo | Name        | Birthdate  | Age | Gender | Address code |
|------------|-------------|------------|-----|--------|--------------|
| 1          | Miller, Tom | 33.01.2001 | 15  | M      | NULL         |
| 1          | John Mayor  | 15.01.2001 | 15  | M      | 98144        |
| 2          | Mrs. Bush   | 31.10.1988 | 22  | Q      | 00000        |
| 3          | Martin      | 31.10.1988 | 22  | M      | 7589         |



# Extraction in ETL

Correcting the data

Automatically during ETL

*E.g., address of a customer if a correct reference table exists*

Manually after ETL is finished

*ETL stored bad data in error log tables or files*

*ETL flags bad data (e.g. invalid)*

In the source systems

*Correcting the data at the source is best approach but slow and often not feasible*

# Extraction in ETL

## Dummy dimension data

### Handling nulls

*Missing values can represent*

an unknown value - like date of birth of a customer

a missing value - like engine\_id for a car (logical not null constraint)

### Handling inaccuracies

*Wrong data – like date of 32.10.2022*

| DimensionTable_X | Description |
|------------------|-------------|
| -1               | Unknown     |
| -2               | Missing     |
| -3               | Invalid     |

# Data Quality

## Defining Data Quality

### Correct

*data describe their associated objects truthfully and faithfully*

### Unambiguous

*The values and descriptions in data can be taken to have only one meaning*

### Consistent

*The values and descriptions in data use one constant notational convention to convey their meaning*

### Complete

*Individual - ensuring that the individual values and descriptions in data are defined*

*Aggregate - makes sure that you didn't somehow lose records altogether somewhere in the information flow.*

# Data Quality

## Quality Screens

a set of diagnostic filters  
each implement a test in the data flow

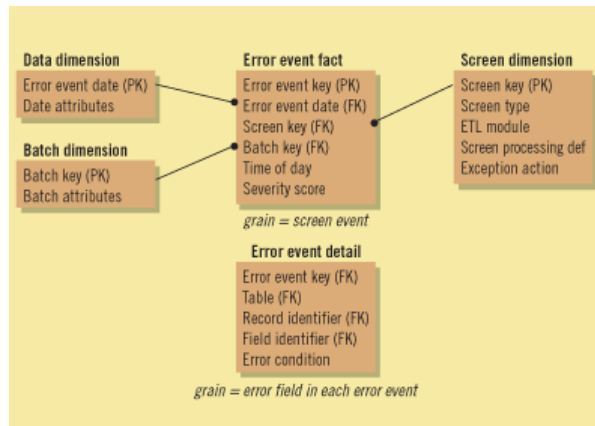
*if it fails it records an error in the Error Event Schema*

## Error Event Schema

place, where all error events thrown by quality screens, are recorded

holds information about exactly when the error occurred and the severity of the error

maintained in multi-dimensional structure



# Extraction

## Example – Data Sampling

The simplest way to check for basic anomalies is to count the rows in a table while grouping on the column in question

This simple query reveals the distribution of values and displays potentially corrupt data

```
select state, count(*)  
from order_detail  
group by state
```

| State         | Count(*) |
|---------------|----------|
| Rhode Island  | 1        |
| Mississippi   | 2        |
| New Yourk     | 5        |
| Connecticut   | 7        |
| New Mexico    | 43,844   |
| Vermont       | 64,547   |
| Mississippi   | 78,198   |
| Utah          | 128,956  |
| Wyoming       | 137,630  |
| Missouri      | 148,953  |
| Rhode Island  | 182,067  |
| Minnesota     | 195,197  |
| North Dakota  | 203,286  |
| Michigan      | 241,245  |
| Washington    | 274,528  |
| Pennsylvania  | 287,289  |
| Montana       | 337,128  |
| Louisiana     | 341,972  |
| Virginia      | 346,691  |
| West Virginia | 359,848  |
| Delaware      | 422,091  |
| Iowa          | 456,302  |
| Massachusetts | 480,958  |
| Tennessee     | 483,688  |
| New York      | 494,332  |
| Nevada        | 506,235  |
| South Dakota  | 514,485  |
| Indiana       | 553,267  |
| Connecticut   | 562,436  |

# Data Quality

Quality screens are divided into three categories:

Column screens.

*Used to test individual column,*

e.g. for unexpected values like NULL values; non-numeric values that should be numeric; out of range values; etc.

Structure screens.

*Used to test for the integrity of different relationships between columns (typically foreign/primary keys) in the same or different tables.*

*Used for testing that a group of columns is valid according to some structural definition it should adhere.*

Business rule screens.

*Used to test whether data, maybe across multiple tables, follows specific business rules*

e.g. if a customer is marked as a certain type of customer, the business rules that define this kind of customer should be adhered

# Screen Example

## Known Table Row Counts

In some cases the number of records to be expected of a given data type from a given data provider is known.

The known table record count case can be handled by simple screen SQL, such as the following:

```
SELECT COUNT(*)  
FROM work_in_queue_table  
WHERE source_system_name =  
'Source System Name'  
HAVING COUNT(*) <>  
'Known_Correct_Count'
```

## Column Length Restriction

Screening on the length of strings in textual columns is useful in both staged and integrated record errors.

An example of a SQL SELECT that performs such a screening:

```
SELECT  
unique_identifier_of_offending_records  
FROM work_in_queue_table  
WHERE source_system_name =  
'Source System Name'  
AND LENGTH(numeric_column) IS  
NOT BETWEEN min AND max.
```

# Screens

## Other examples

### Column Numeric and Date Ranges

*from a data-quality perspective data may have ranges of validity that are restrictive*

### Column Explicit Valid Values

*A given column may have a set of known discrete valid values as defined by its source system*

### Column Explicit Invalid Values

*A given column is routinely populated with values known to be incorrect and for which there is no known set of discrete valid values,  
explicitly screen for these invalid values.*

### Checking Table Row Count Reasonability

*This class of screens is quite powerful but a bit more complex to implement.*

### Checking Column Distribution Reasonability

*The ability to detect when the distribution of data across a dimensional attribute has strayed from normalcy is another powerful screen.*



# Cleaning and conforming data are critical ETL system tasks

these are the steps where the ETL system adds value to the data

provides guidance whether data can be used for its intended purposes

actually changes data

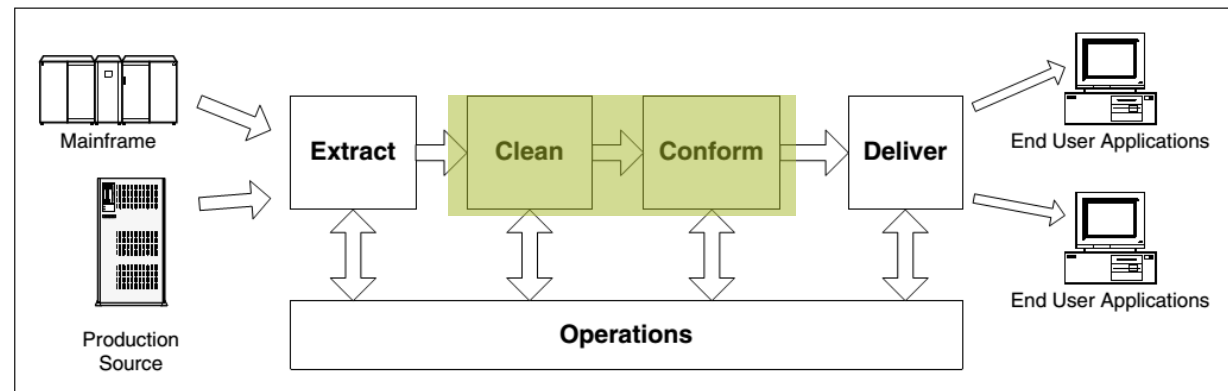
## The cleaning and conforming steps generate potent metadata

Looking backward toward the original sources

*this metadata is a diagnosis of what's wrong in the source systems*

Ultimately, dirty data can be fixed only by changing the way these source systems collect data

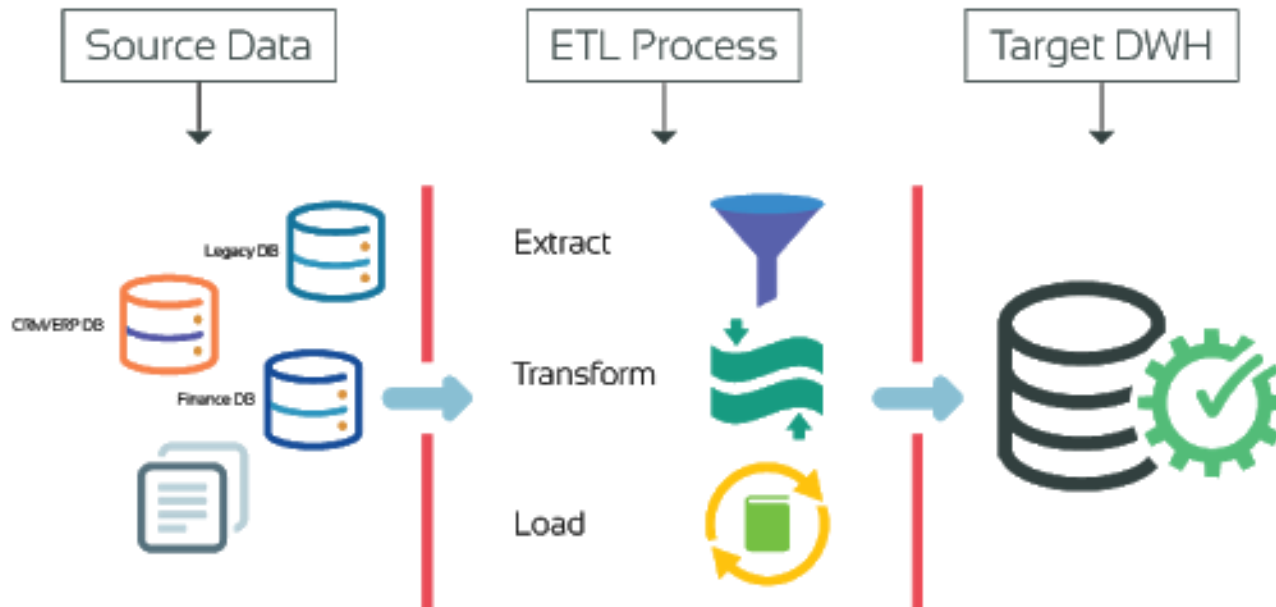
# Transformation



# Transformation

In general.

Fix the data downstream



# Transformation

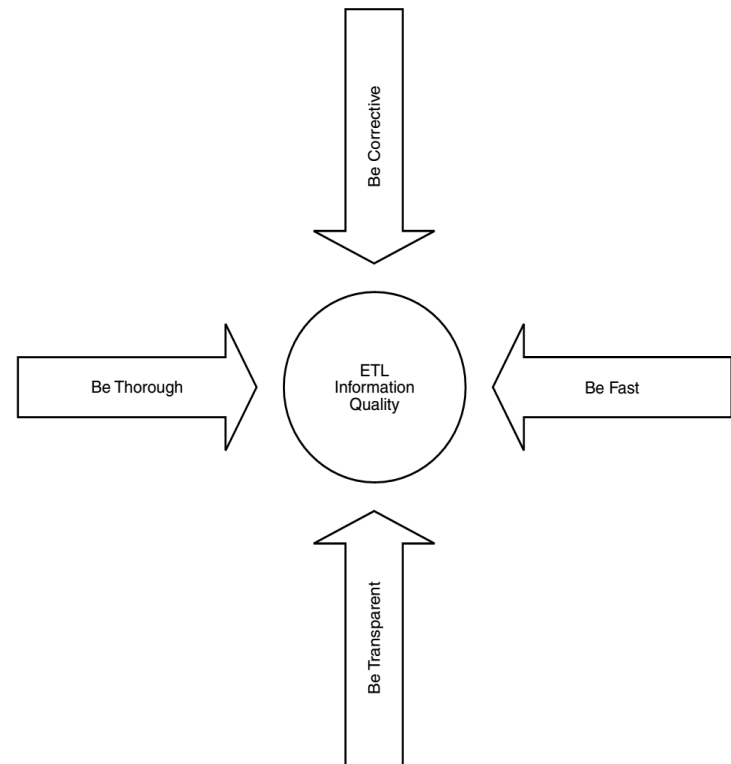
## Conflicting priorities

### Completeness vs speed

*The data-quality ETL cannot be optimized for both speed and completeness*

### Corrective vs transparent

*The data-cleaning process is often expected to fix dirty data, yet at the same time provide a clear and accurate view of the data as it was captured*



# Clean

## Data Transformation

### Smoothing:

*remove noise from data*

### Aggregation:

*summarization, data cube construction*

### Normalization:

*scaled to fall within a small, specified range*

### Generalization:

*concept hierarchy climbing*

### Attribute/feature construction

*New attributes constructed from the given ones*

# Transformation

## Common Transformations

Data type conversions

*ASCII/Unicode*

*String manipulations*

*Date/time format conversions*

e.g., Unix time 1201928400

Normalization/denormalization

*To the desired DW format*

*Depending on source format*

Building keys

*Table matches production keys to surrogate DW keys*

*Correct handling of history*

## Transformation

is applying any business rules to the data to meet reporting requirements

changes the raw data to the correct reporting formats

## Accuracy

Data cannot be dropped or changed in a way that corrupts its meaning.

Every data point should be auditable at every stage in your process.

## Transform data

removing extraneous or erroneous data (cleaning), applying business rules, checking data integrity (ensuring that the data was not corrupted in source, or corrupted by ETL, and that no data was dropped in previous stages), and creating aggregates as necessary

For example

*analysing revenue – you can summarize the dollar amount of invoices into a daily or monthly total.*

Need a series of rules or functions that can achieve the required transformations

*design and test*

*run on the extracted data*

# The clean step involves:

*Enforcing column properties*

*Enforcing structure*

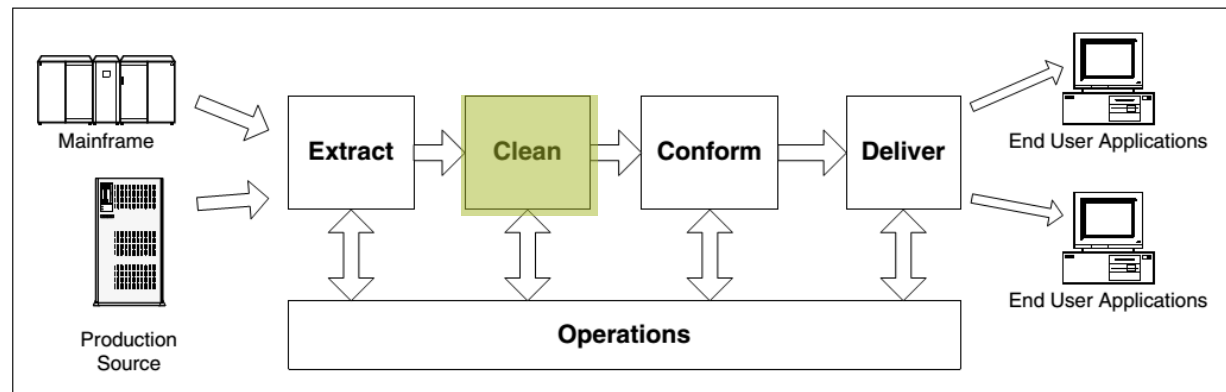
*Enforcing data and value rules*

*Enforcing complex business rules*

*Building a metadata foundation to describe data quality*

*Staging the cleaned data to disk*

# Data Flow



# Clean

## Cleansing subsystem

comprehensive architecture for cleansing data, capturing data quality events, as well as measuring and ultimately controlling data quality in the data warehouse

## Goals for the subsystem should include:

- Early diagnosis and triage of data quality issues

- Requirements for source systems and integration efforts to supply better data

- Provide specific descriptions of data errors expected to be encountered in ETL

- Framework for capturing all data quality errors and precisely measuring data quality metrics over time

- Attachment of quality confidence metrics to final data

## Major Tasks

### Data cleansing

*Fill in missing values, smooth noisy data, identify or remove outliers, and resolve inconsistencies*



# Clean

## Quality Screens

a set of quality screens that act as diagnostic filters in the data flow pipelines – each quality screen is a test:

*If the test against the data is successful, nothing happens and the screen has no side effects.*

*But if the test fails, then it must drop an error event row into the error event schema and choose to either halt the process, send the offending data into suspension, or merely tag the data.*

three categories of data quality screens:

*column screens*

test the data within a single column

*structure screens*

test the relationship of data across columns

*business rule screens*

more complex tests that do not fit the simpler column or structure screen categories

# Clean

Each quality screen has to decide what happens when an error is thrown:

- halting the process;

- sending the offending record(s) to a suspense file for later processing;

- tagging the data and passing it through to the next step in the pipeline;

# Clean

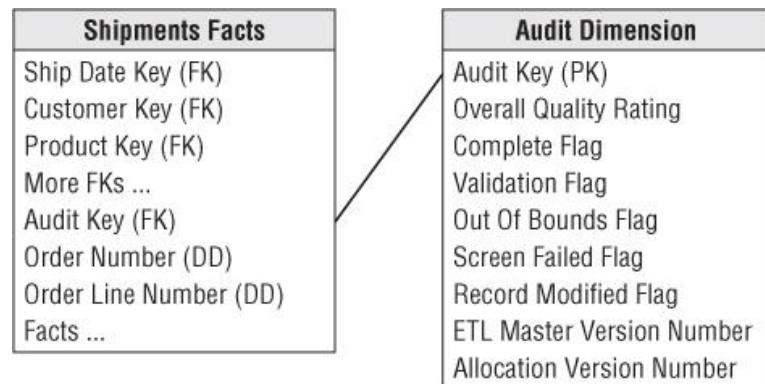
If a perfect run is not feasible

A nice idea is to include audit dimension

Audit dimension

a special dimension that is assembled in the back room by the ETL system for each fact table

contains the metadata context at the moment when a specific fact table row is created



# Clean

## Deduplication

Often dimensions are derived from several sources

*Sometimes, the data can be matched through identical values in some key column.*

Even when a definitive match occurs, other columns in the data might contradict one another, requiring a decision on which data should survive.

Deduplication ensures that one accurate record exists for each business entity represented in a analytic database

### Example

|                |     |              |             |    |       |
|----------------|-----|--------------|-------------|----|-------|
| Tim Jones      | 123 | Main Street  | Marlboro    | MA | 12234 |
| T. Jones       | 123 | Main St.     | Marlborough | MA | 12234 |
| Timothy Jones  | 321 | Maine Street | Marlborog   | AM | 12234 |
| Jones, Timothy | 123 | Maine Ave    | Marlborough | MA | 13324 |

The sales for around \$500 are counted for each tuple.

**Is it the same person?**

# Clean

## Deduplication

Unfortunately, there is seldom a universal column that makes the merge operation easy

*data may need to be evaluated on different fields to attempt a match  
sometimes, a match may be based on fuzzy criteria*

## Survivorship

*is the process of combining a set of matched records into a unified image that combines the highest quality columns from the matched records into a conformed row.*

*involves establishing clear business rules that define the priority sequence for column values from all possible source systems to enable the creation of a single row with the best-survived attributes*

# Clean

Handling Missing Data:

Ignore the tuple:

*usually done only when the class label is missing*

Fill in the missing value manually

*tedious + infeasible?*

Fill in it automatically with

*a global constant : e.g., “unknown”*

*a new class*

*the attribute mean*

*the attribute mean for all samples belonging to the same class:*

*smart ?*

*the most probable value:*

*inference-based such as Bayesian formula or decision tree*

# Clean

## Handling Noisy Data:

### Binning method:

*first sort data and partition into (equi-depth) bins*

*then one can smooth by bin means, smooth by bin median, smooth by bin boundaries, etc.*

### Clustering

*detect and remove outliers*

### Combined computer and human inspection

*detect suspicious values and check by human (e.g., deal with possible outliers)*

### Regression

*smooth by fitting the data into regression functions*

# Clean

Smoothing using simple discretization methods:

## Binning

*Equal-width (distance) partitioning:*

Divides the range into N intervals of equal size:

uniform grid

if A and B are the lowest and highest values of the attribute, the width of intervals will be:  $W = (B - A)/N$ .

The most straightforward, but outliers may dominate presentation

Skewed data is not handled well.

*Equal-depth (frequency) partitioning:*

Divides the range into N intervals, each containing approximately same number of samples

Good data scaling

Managing categorical attributes can be tricky.



# Clean

## Binning Methods for Data Smoothing

Sorted data for price (in dollars):

4, 8, 9, 15, 21, 21, 24, 25, 26, 28, 29, 34

Partition into (equi-depth) bins:

*Bin1:* 4, 8, 9, 15

*Bin2:* 21, 21, 24, 25

*Bin3:* 26, 28, 29, 34

Smoothing by bin means:

*Bin1:* 9, 9, 9, 9

*Bin2:* 23, 23, 23, 23

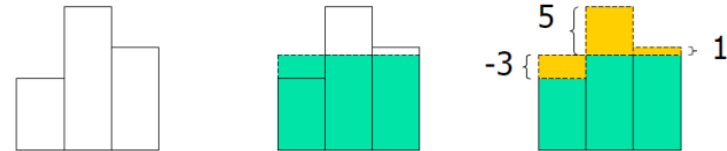
*Bin3:* 29, 29, 29, 29

Smoothing by bin boundaries:

*Bin1:* 4, 4, 4, 15

*Bin2:* 21, 21, 25, 25

*Bin3:* 26, 26, 26, 34



# Clean

## Automatic Data Cleansing

### Statistical

*Identifying outlier fields and records using the values of mean, standard deviation, range, etc  
Confidence intervals are taken into consideration for each field*

### Pattern Based

*Identify outlier fields and records that do not conform to existing patterns in the data  
A pattern is defined by a group of records that have similar characteristics*

### Clustering

*Identify outlier records using clustering based on Euclidian (or other) distance  
The main drawback of this method is computational time*

### Association Rules

*Association rules with high confidence and support define a different kind of pattern*

### Reference data

*create a set of data that defines the set of permissible values your data may contain.  
For example, in a country data field, you can define the list of country codes allowed.*

## conform step includes:

*Conforming business labels (in dimensions)*

*Conforming business metrics and performance indicators (in fact tables)*

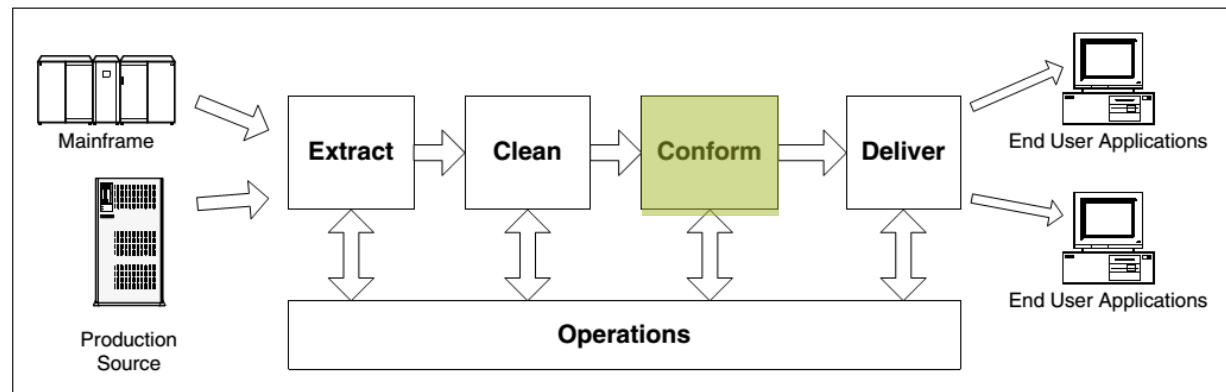
*Standardization*

*Deduplicating*

*Internationalizing*

*Staging the conformed data to disk*

# Data Flow

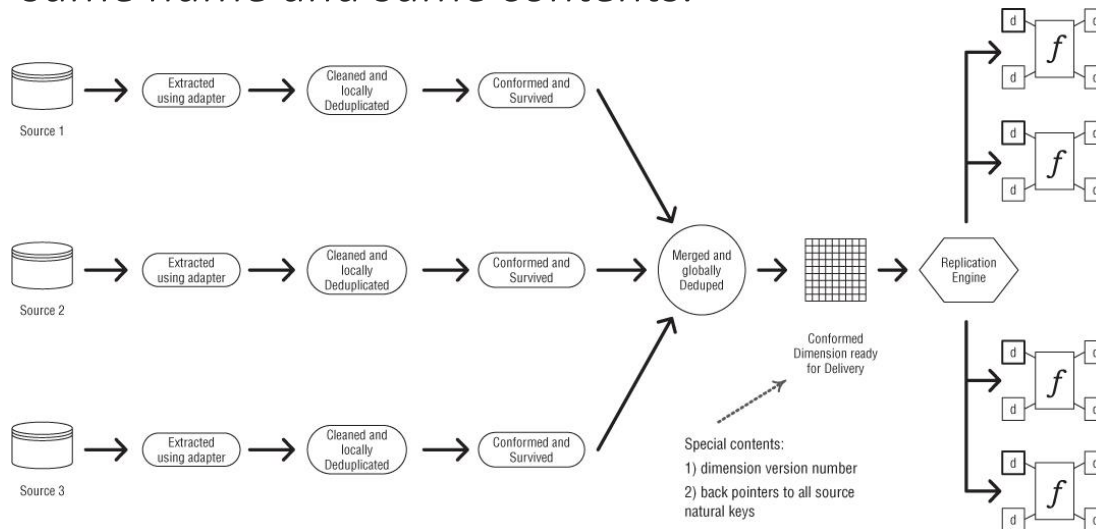


# Conform

## Conforming dimensions

consists of all the steps required to align the content of some or all the columns in a dimension with columns in similar or identical dimensions in other parts of the data warehouse

*Reminder – for two dimensions to be conformed, they must share at least one common attribute with the same name and same contents.*



# Conform

## Data integration

- Several conceptual schemas need to be combined into a unified global schema

- All differences in perspective and terminology have to be resolved

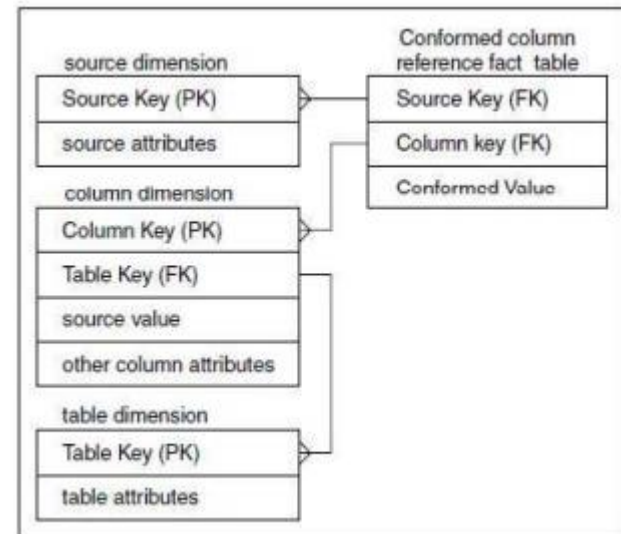
- All redundancy has to be removed

# Conformed attributes

## Implementing conforming modules

Use of auxiliary tables (see diagram)

*Mapping of attribute values between individual systems*



# Conform

## Major Tasks

### Data transformation

*Standardization/normalization*

*Aggregation*

*Enrichment*

### Data reduction

*Obtains reduced representation in volume but produces the same or similar analytical results*

# Conform

## Typical Techniques:

- Conversion and standarization methods

  - (date formats \dd/mm/yyyy", names conventions: Jan Kowalski),*

- Parsing text fields in order to identify and isolate data elements:

  - Transformation (splitting the text into records { title = mgr, first name = Jan, last name = Kowalski } ),*

  - Standardization (Jan Kowalski, magister ) mgr Jan Kowalski),*

- Dictionary-based methods

  - (database of names, geographical places, pharmaceutical data),*

- Domain-specific knowledge methods to complete data

  - (postal codes),*

- Rationalization of data

  - (PHX323RFD110A4 ) Print paper, format A4),*

- Rule-based cleansing

  - (replace gender by sex)*

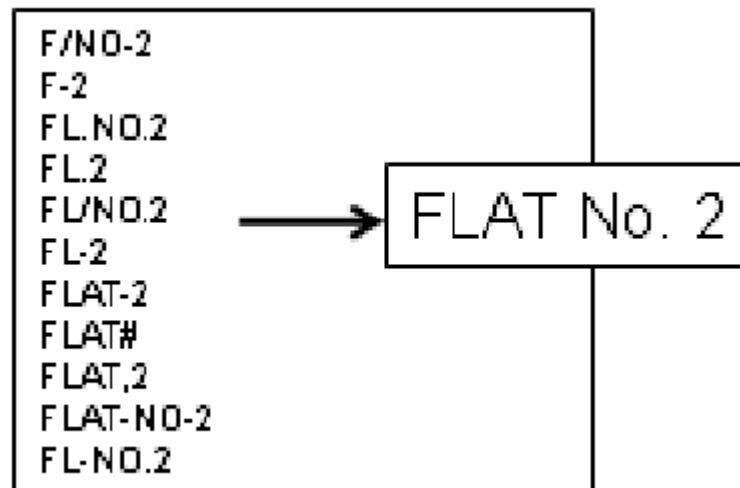
- Cleansing by using data mining.



# Conform

## Conversion

Convert and standardize different data formats



# Conform

## Enrichment

### Input Data

HAJI MUHAMMAD IBRAHIM, GOVT. CONT.  
K. S. ABDULLAH & BROTHERS,  
MAMOOJI ROAD, ABDULLAH MANZIL  
RAWALPINDI, Ph 67855



### Parsed Data

|                       |                           |
|-----------------------|---------------------------|
| <b>First Name:</b>    | HAJI MUHAMMAD             |
| <b>Family Name:</b>   | IBRAHIM                   |
| <b>Title:</b>         | GOVT. CONT.               |
| <b>Firm:</b>          | K. S. ABDULLAH & BROTHERS |
| <b>Firm Location:</b> | ABDULLAH MANZIL           |
| <b>Road:</b>          | MAMOOJI ROAD              |
| <b>Phone:</b>         | 051-67855                 |
| <b>City:</b>          | RAWALPINDI                |
| <b>Code:</b>          | 46200                     |

# Conform

## Discretization

Three types of attributes:

*Nominal — values from an unordered set*

*Ordinal — values from an ordered set*

*Continuous — real numbers*

Discretization:

*divide the range of a continuous attribute into intervals*

*Some classification algorithms only accept categorical attributes.*

*Reduce data size by discretization*

*Prepare for further analysis*

# Conform

## Discretization and Concept hierarchy

### Discretization

*reduce the number of values for a given continuous attribute by dividing the range of the attribute into intervals. Interval labels can then be used to replace actual data values*

### Concept hierarchies

*reduce the data by collecting and replacing low level concepts (such as numeric values for the attribute age) by higher level concepts (such as young, middle-aged, or senior)*

# Conform

## Normalization

- min-max normalization

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new\_max}_A - \text{new\_min}_A) + \text{new\_min}_A$$

- z-score normalization

$$v' = \frac{v - \text{mean}_A}{\text{stddev}_A}$$

- normalization by decimal scaling

$$v' = \frac{v}{10^j} \quad \text{Where } j \text{ is the smallest integer such that } \max(|v'|) < 1$$

# Conform

## Conflicts and Dirty Data

- Different logical models of operational sources,
- Different data types (account number stored as String or Numeric),
- Different data domains (gender: M, F, male, female, 1, 0),
- Different date formats (dd-mm-yyyy or mm-dd-yyyy),
- Different field lengths (address stored by using 20 or 50 chars),
- Different naming conventions: homonyms and synonyms,

## Semantic conflicts

- when the same objects are modelled on different logical levels,

## Structural conflicts

- when the same concepts are modelled using different structures.

# Data delivery from the ETL system includes:

*Loading flat and snowflaked dimensions (Loading subdimensions)*

*Generating time dimensions*

*Conforming dimensions and conforming facts*

*Loading text facts in dimensions*

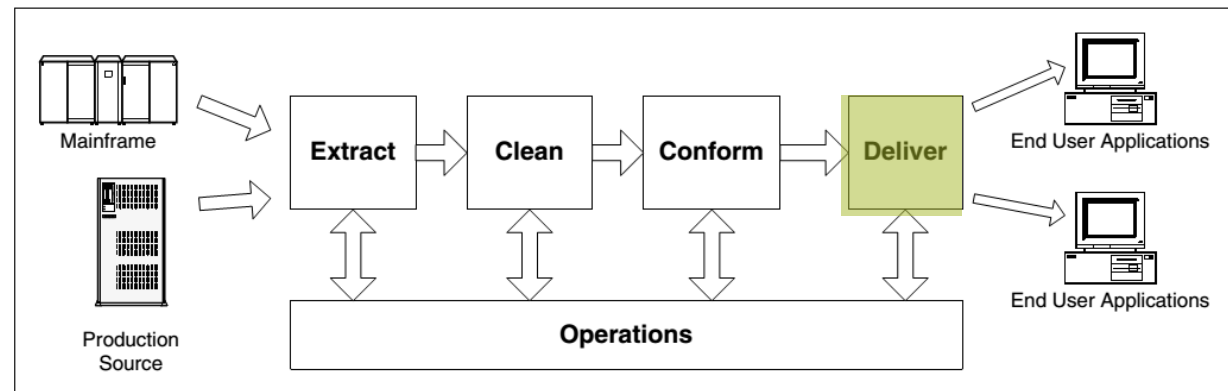
*Running the surrogate key pipeline for fact tables*

*Loading fundamental fact table grains*

*Loading and updating aggregations*

*Staging the delivered data to disk*

# Data Flow



# Load

The primary mission of the ETL system is the handoff of the dimension and fact tables in the delivery step

## Load of Data (Delivering)

Delivering is the final essential ETL step

*data must be loaded into the warehouse.*

*cleaned and conformed data is written into the dimensional structures*

Loading the warehouse includes some other processing tasks:

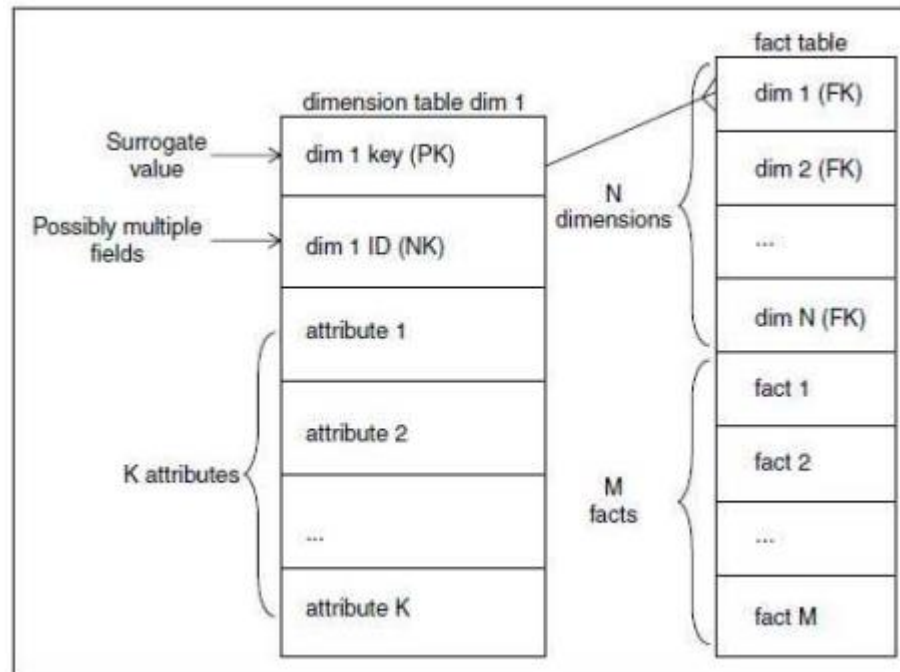
*Checking integrity constraints, sorting, summarizing, creating indexes, etc.*

Batch load utilities are used for loading.

*A load utility must allow to monitor status, to cancel, suspend, and resume a load, and to restart after failure with no loss of data integrity*



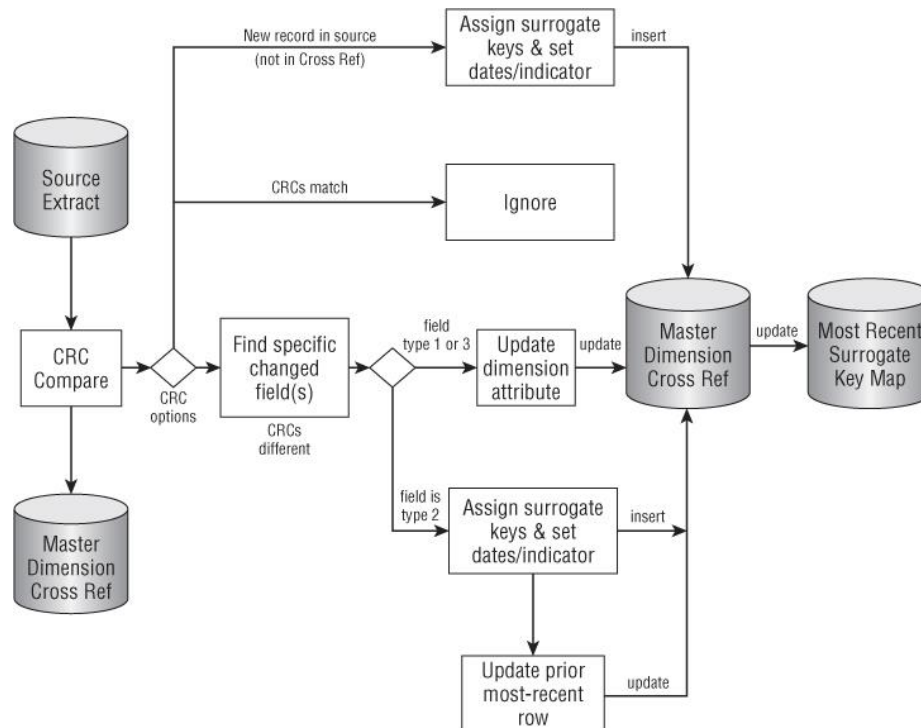
# Load



# Load

## Slowly Changing Dimension

Important elements of the ETL architecture is the capability to implement slowly changing dimension (SCD) logic



# Load

## Surrogate keys

It is recommended the use of surrogate keys for all dimension tables

### Surrogate key generator

*independently generate surrogate keys for every dimension; it should be independent of database instance and able to serve distributed clients*

*generate a meaningless key, typically an integer, to serve as the primary key for a dimension row*

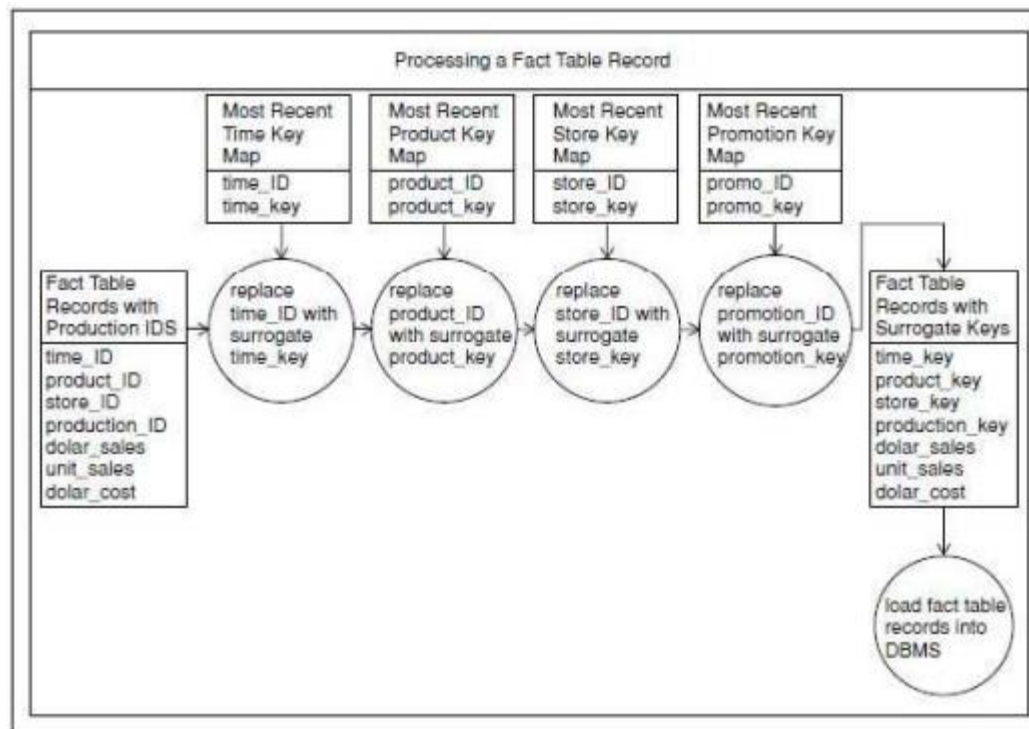
### Keys

*Leaving it to DB triggers affects performance and lost of control*

*For improved efficiency, consider having the ETL tool generate and maintain the surrogate keys*

*Avoid concatenating the operational key of the source system and a date/time stamp.*

# Surrogate key – sample pipeline



# Load

## Handling hierarchies

### Fixed depth hierarchies

*Determine proper relations with the key*

*Determine proper key for each level*

### Ragged hierarchies (unbalanced)

*Use proper technique to model*

fixing number of levels

bridge

### Consider using snowflake

## Handling special dimensions

### Date/Time dimension

*Permanent dimension*

Should be done once – during initial etl

### Junk dimension

*two alternatives:*

building all valid combinations in advance

recognizing and creating new combinations on-the-fly

### Role-playing dimension

### Mini-dimension

*Similar handling to junk dimension*

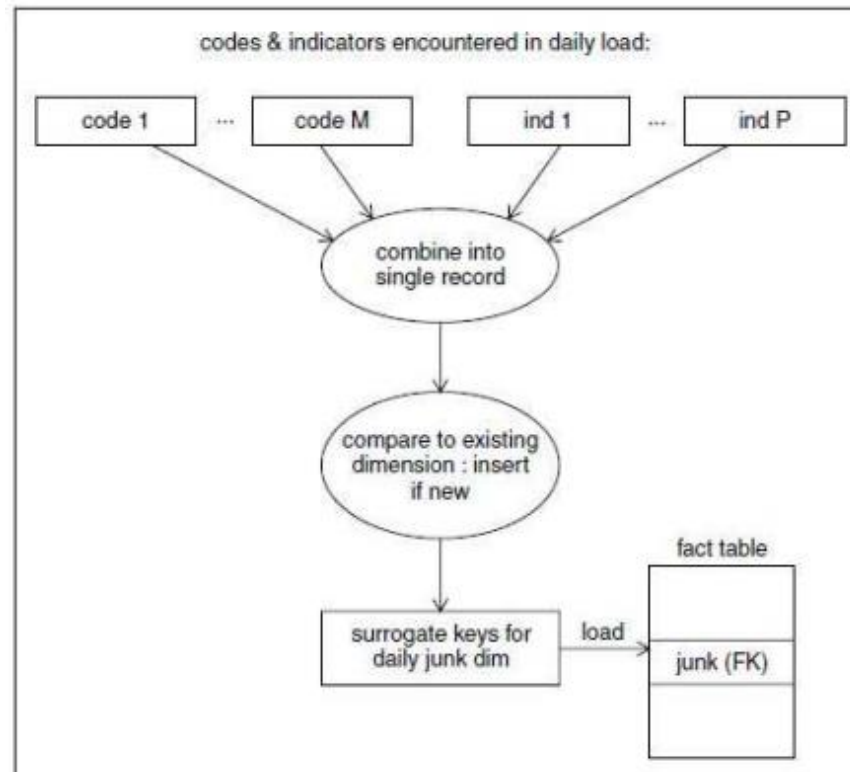
### Shrunk subset dimension

*build conformed shrunk dimensions from the base dimension to assure conformance*

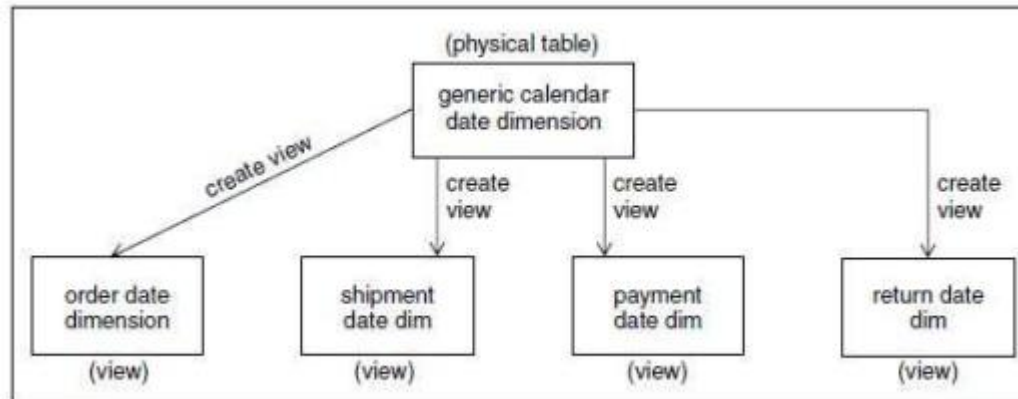
### Small static dimension

*created entirely by the ETL system without a real outside source*

# Junk dimension – sample pipeline



# Role-playing dimension





# Load

## Significance of Data Loading Strategies

### Data Freshness

*Very fresh low update efficiency*

*Historical data, high update efficiency*

*Always trade- offs in the light of goals*

### System performance

*Availability of staging table space*

*Impact on query workload*

### Data Volatility

*How much data is changed within the window of the refresh*

Ratio of new to historical data

High percentages of data change (batch update)

# Load

## Load of Data Issues

- Huge volumes of data to be loaded

  - Sequential loads can take a very long time*

- Small time window available when warehouse can be taken off-line

  - (usually nights)*

- When to build index and aggregated tables

- Allow system administrators to monitor, cancel, resume, change load rates

- Recover gracefully - restart after failure from where you were and without loss of data integrity

  - Using checkpoints ensures that if a failure occurs during the load, the process can restart from the last checkpoint*

# Why preprocess ?

No quality data, no quality analysis !

Quality decisions must be based on quality data

*e.g., duplicate or missing data may cause incorrect or even misleading statistics*

Serious problems due to dirty data

*Decisions taken at government level using wrong data resulting in undesirable results*

Data warehouse needs consistent integration of quality data

Data extraction, cleaning, and transformation comprises the majority of the work of building a data warehouse

*Bill Inmon*

# ETL Environment

## Managing the ETL Environment

ETL system must constantly work toward fulfilling three criteria:

### *Reliability.*

processes must consistently run, run to completion to provide data on a timely basis that is trustworthy at any level of detail.

### *Availability.*

data warehouse must meet its service level agreements (SLAs).

The warehouse should be up and available as promised.

### *Manageability.*

data warehouse is never done, it constantly grows and changes along with the business

ETL processes need to gracefully evolve as well.

# ETL Environment

## Managing the ETL Environment

### Robust ETL scheduler

*much more than just launching jobs on a schedule - needs to be aware of and control the relationships and dependencies between ETL jobs*

*must also capture metadata regarding the progress and statistics of the ETL process during its execution*

*support a fully automated process, including notifying the problem escalation system in the event of any situation that requires resolution*

### Backup, recovery, restart

*goal is to allow the data warehouse to get back to work after a failure*

*includes backing up the intermediate staging data necessary to restart failed ETL jobs*

### Monitor performance

### Security and compliance

### Metadata

# Detailed information

Data quality screening

# Data Quality

## Data validation

an automated process confirms whether data pulled from sources has the expected values - for example, in a database of financial transactions from the past year, a date field should contain valid dates within the past 12 months.

The validation engine rejects data if it fails the validation rules.

You analyze rejected records, on an ongoing basis, to identify what went wrong, correct the source data, or modify extraction to resolve the problem in the next batches.

# Data Quality

## Data Quality - Process Flow

The screening technique

*is a data warehouse data quality technique which uses the inside-out approach (data -> issue)*

A series of data-quality screens or error checks are queued for running

*run the rules - defined in metadata*

*for highest performance processing stream invokes waves of screens that can be run in parallel*

## Quality Screens

*a set of diagnostic filters*

*each implement a test in the data flow*

*if it fails it records an error in the Error Event Schema*



# Data Quality

## Data Quality - Process Flow

As each screen is run

*each occurrence of errors encountered is recorded  
in an error event record*

The calling process

*waits for each wave of screens to complete  
before invoking the next wave of screens  
runs until there are no more screen waves left to run*

Before enabling a screening module

*it is advisable to use data profiling - to discover the true content,  
structure and quality of data of a data source.*

# Data Quality

## Data Quality - Process Flow

The driving engine of the screening technique consists of a table containing a number of data quality screens

*each screen acts as a constraint or data rule*

*filters the incoming data by testing one specific aspect of quality*

## Error Event Schema

*is a centralized dimensional schema whose purpose is to record every error event thrown by a quality screen anywhere in the ETL pipeline*

*holds information about exactly when the error occurred and the severity of the error*

*maintained in multi-dimensional structure*

# Data Quality

Error Event Schema consists of:

Error Event Fact table

*time and severity score*

Date dimension table

*(when),*

Batch job dimension table

*(where)*

Screen dimension table

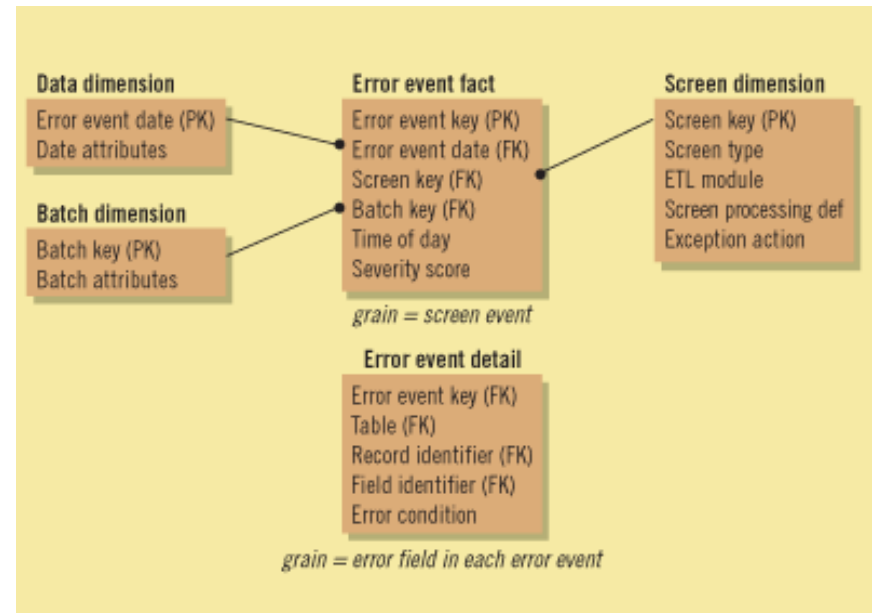
*(who produced error)*

Error Event Detail fact table

*with a foreign key to the main table*

*contains detailed information*

*in which table, record and field the error occurred and the error condition*



# Data Quality

## Data Quality - Process Flow

When each of the data-quality checks has been run

*the error event fact table is queried for fatal events encountered during the overall data-quality process*

*If none are found*

normal ETL processing continues

*otherwise*

a halt condition is returned to the overall calling ETL process

an orderly shutdown of the overall ETL process is performed

# Data Quality

## Data Quality - Process Flow

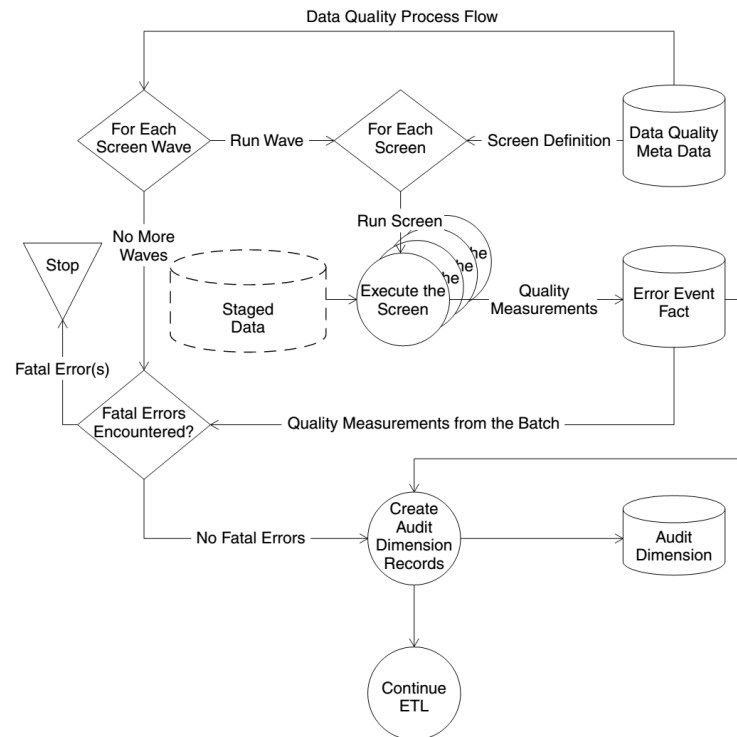
Based on the findings of these screens, the ETL job stream can choose to:

*Pass the record with no errors*

*Pass the record, flagging offending column values*

*Reject the record*

*Stop the ETL job stream*



# Data Quality

Quality screens are divided into three categories:

## Column screens

*Used to test individual column,*

e.g. for unexpected values like NULL values; non-numeric values that should be numeric; out of range values; etc.

## Structure screens

*Used to test for the integrity of different relationships between columns (typically foreign/primary keys) in the same or different tables.*

*Used for testing that a group of columns is valid according to some structural definition it should adhere.*

## Business rule screens

*Used to test whether data, maybe across multiple tables, follows specific business rules*

e.g. if a customer is marked as a certain type of customer, the business rules that define this kind of customer should be adhered

# Screens

## Known Table Row Counts

In some cases, the information-quality leader absolutely knows, through business policy, the number of records to be expected of a given data type from a given data provider.

The known table record count case can be handled by simple screen SQL, such as the following:

```
SELECT COUNT(*)  
FROM work_in_queue_table  
WHERE source_system_name = 'Source System Name'  
HAVING COUNT(*) <> 'Known_Correct_Count'
```

# Screens

## Column Nullity

The determination of which columns are required (versus allowed to be null) in data records is very important and typically varies by source system

In dimensional models, integrated records often have more restrictive nullity rules than source data, because nearly all dimensional attribute columns are required to be populated

The proposed approach for testing nullity is to build a library of source specific nullity SQL statements that return the unique identifiers of the offending rows, such as the following:

```
SELECT unique_identifier_of_offending_records  
FROM work_in_queue_table  
WHERE source_system_name = 'Source System Name'  
AND column IS NULL
```



# Screens

## Column Numeric and Date Ranges

from a data-quality perspective data may have ranges of validity that are restrictive

*data-cleaning subsystem should be able to detect and record instances of numeric columns that contain values that fall outside of valid ranges*

An example of a SQL SELECT statement to screen these potential errors follows:

```
SELECT unique_identifier_of_offending_records  
FROM work_in_queue_table  
WHERE source_system_name = 'Source System Name'  
AND numeric_column IS NOT BETWEEN min AND max
```

# Screens

## Column Length Restriction

Screening on the length of strings in textual columns is useful in both staged and integrated record errors.

An example of a SQL SELECT that performs such a screening:

```
SELECT unique_identifier_of_offending_records  
FROM work_in_queue_table  
WHERE source_system_name = 'Source System Name'  
AND LENGTH(numeric_column) IS NOT BETWEEN min AND max.
```

# Screens

## Column Explicit Valid Values

A given column may have a set of known discrete valid values as defined by its source system

Therefore, a representative SQL statement might be:

```
SELECT unique_identifier_of_offending_records  
FROM work_in_queue_table Q  
WHERE source_system_name = 'Source System Name'  
AND column NOT EXISTS  
  ( SELECT anything  
    FROM column_validity_reference_table  
    WHERE column_name = "column_name"  
    AND source_system_name = 'Source System Name'  
    AND valid_column_value = Q.column_value  
  )
```

# Screens

## Column Explicit Invalid Values

A given column is routinely populated with values known to be incorrect and for which there is no known set of discreet valid values,

*explicitly screen for these invalid values.*

The explicit invalid values screen should obviously not attempt to exhaustively filter out all possible invalid values—just pick off the frequent offenders.

*Other data-cleaning technologies, such as name and address standardization and matching, are far more appropriate for these tasks.*

An example that follows hard-codes the offending strings into the screen's SQL statement.

```
SELECT unique_identifier_of_offending_records
FROM work_in_queue_table
WHERE source_system_name = 'Source System Name'
AND UPPER(column) IN ("UNKNOWN", "?", list of other frequent offenders... )
```

A slightly more elegant approach might compare the data values to a table full of frequent offenders

# Screens

## Checking Table Row Count Reasonability

This class of screens is quite powerful but a bit more complex to implement.

It attempts to ensure that the number of rows received from a data source is reasonable

*meaning that the row counts fall within a credible range based on previously validated record count histories.*

- calculating the number of standard deviations a value falls from the mean of previous similar values

- use more advanced and professional value predictors

# Screens

Checking Table Row Count Reasonability

```
SELECT
    AVERAGE(number_of_records)-3*STDDEV(number_of_records),
    AVERAGE(number_of_records)+3*STDDEV(number_of_records)
INTO min_reasonable_records, max_reasonable_records
FROM data_staging_table_record_count
WHERE source_system_name = 'Source System Name'
```

```
SELECT
    COUNT(*)
FROM work_in_queue_table
WHERE source_system_name = 'Source System Name'
HAVING COUNT(*) NOT BETWEEN min_reasonable_records AND
max_reasonable_records
```

# Screens

## Checking Column Distribution Reasonability

The ability to detect when the distribution of data across a dimensional attribute has strayed from normalcy is another powerful screen.

This screen enables you to detect and capture situations when a column with a discrete set of valid values is populated with a data distribution that is skewed abnormally

# Screens

And many other ...

## Advantages of the screening technique

- Framework for capturing all data quality errors

- Framework for measuring data quality over time

- Easily integrated with existing metadata

- Reusable

- Validations, which are no longer needed, can be easily (logically) removed

- New validations – screens - can be added easily

- Possibility to qualify severity

- One central place for all errors

- Error traceability - backward traceability up to the delivered flat files or sources

- Possibility for trend analysis or other analytics via the star schema



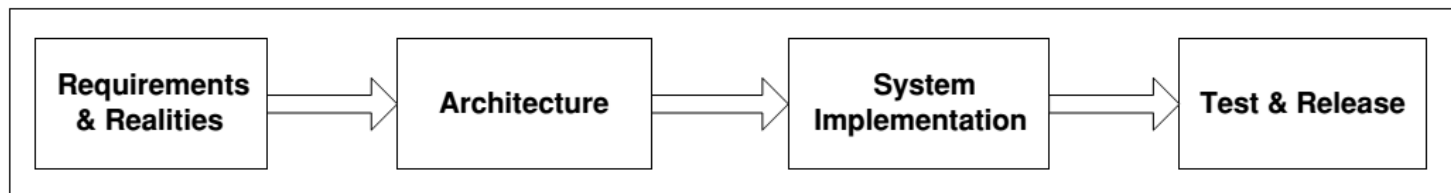
# Detailed information

ETL System Design

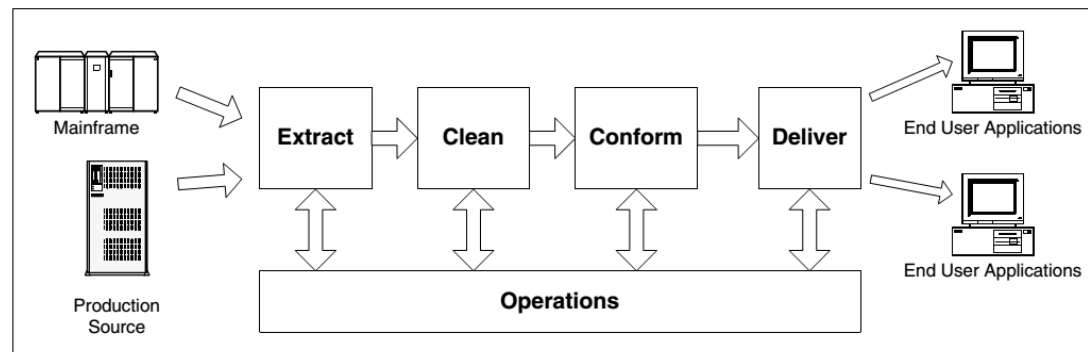
# ETL System

Two simultaneous threads must be kept in mind when building an ETL system:

the **Planning & Design** thread



and the **Data Flow** thread.



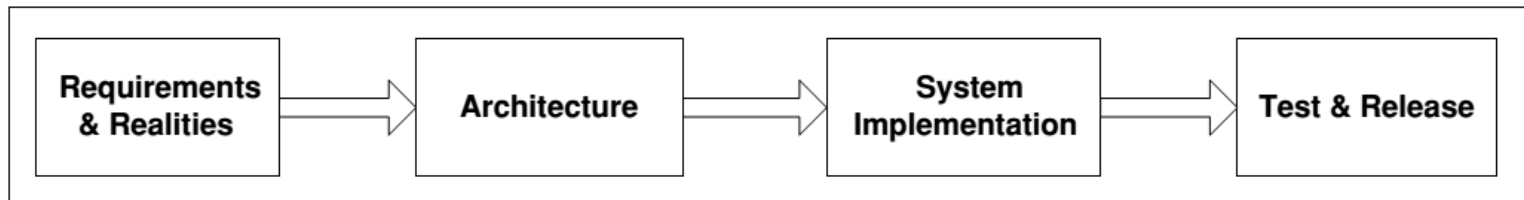
# Planning & Design

## Planning & Design thread

The first step is accounting for all the **requirements and realities**

*These include:*

- Business needs
- Data profiling and other data-source realities
- Compliance requirements
- Security requirements
- Data integration
- Data latency
- Archiving and lineage



# Business needs

From an ETL designer's view

the business needs are the DW/BI system users' information requirements

*Meaning that the information content that business users need to make informed business decisions.*

the business needs directly drive the choice of data sources and their subsequent transformation in the ETL system

*ETL team must understand and carefully examine the business needs.*

# Requirements and Realities

## Requirements and Realities - Compliance Requirements

Laws and regulations

*Several of the financial-reporting issues*

Typical due diligence requirements for the data warehouse include:

*Archived copies of data sources and subsequent staging's of data*

*Proof of the complete transaction flow that changed any data*

*Fully documented algorithms for allocations and adjustments*

*Proof of security of the data copies over time, both on-line and off-line*

# Requirements and Realities

## Requirements and Realities - Data Profiling

is a necessary precursor to designing any kind of system that uses data

*to work with the data, you need to understand their profile*

*it is common to profile the data in advance to identify patterns in the data and determine if there are any data quality issues*

is a systematic examination of the quality, scope, and context of a data source to allow an ETL system to be built

*return metrics from a data set*

*especially relevant to the ETL team who may be handed a data source whose content has not really been vetted*

# Requirements and Realities

## Requirements and Realities - Data Profiling

“[Data profiling] employs analytic methods for looking at data for the purpose of developing a thorough understanding of the content, structure, and quality of the data.

A good data profiling [system] can process very large amounts of data, and with the skills of the analyst, uncover all sorts of issues that need to be addressed.”

[Jack Olson]

# Requirements and Realities

## Requirements and Realities - Data Profiling

At one extreme

*A very clean data source that has been well maintained before it arrives at the data warehouse requires minimal transformation and human intervention to load directly into final dimension tables and fact tables.*



# Requirements and Realities

## Requirements and Realities - Data Profiling

But a dirty data source may require:

*Elimination of some input fields completely*

*Flagging of missing data and generation of special surrogate keys*

*Best-guess automatic replacement of corrupted values*

*Human intervention at the record level*

*Development of a full-blown normalized representation of the data*

# Requirements and Realities

Requirements and Realities - Data Profiling

And at the furthest extreme

*if data profiling reveals that the source data is deeply flawed and cannot support the business' objectives*

the data warehouse effort should be cancelled!

# Requirements and Realities

## Requirements and Realities - Data Integration

- a huge topic for IT

- in many cases, serious data integration must take place among the primary transaction systems of the organization

  - before any of that data arrives at the data warehouse*

- rarely is the data integration complete beforehand

  - unless the organization has a comprehensive and centralized master data management (MDM) system*

    - even then it is likely that other important transaction-processing systems exist outside the main ERP system

  - unless the organization has settled on a single enterprise resource planning (ERP) system*

    - even then archived data might be stored in some legacy systems

# Requirements and Realities

## Requirements and Realities - Data Integration

### Conforming dimensions

*establishing common dimensional attributes across separate databases so that drill across reports can be generated using these attributes*

often textual labels and standard units of measurement need to be established  
however it can be a lot more complex – for instance integrating different hierarchies

### Conforming facts

*agreeing on common business metrics across separate databases so that these numbers can be compared mathematically by calculating differences and ratios*

such as key performance indicators (KPIs)  
but also – remember the what is sales question ?

# Requirements and Realities

## Requirements and Realities - Data latency

requirement describes how quickly the data must be delivered to the end users

*has a huge effect on the architecture and the system implementation*

### Two general approaches

*Traditional batch-oriented data flow*

can be sped up by more clever processing algorithms, parallel processing, and more potent hardware.

*streaming oriented*

if the data latency requirement is sufficiently urgent

# Requirements and Realities

## Requirements and Realities - Archiving and Lineage

every data warehouse needs various copies of old data

*either for comparisons with new data to generate change capture records  
or for reprocessing*

it is recommended to do staging of the data at each point

*where a major transformation has occurred*

when does staging (writing data to disk) turn into archiving  
(keeping data indefinitely on permanent media)?

*all staged data should be archived*

*unless a conscious decision is made that specific data sets will never be  
recovered*

# Requirements and Realities

Requirements and Realities - End User Delivery Interfaces

the final step for the ETL system is the handoff

*to end user applications*

the ETL team, working closely with the modeling team, must take responsibility for the content and the structure of data

*making the end user applications simple and fast*

*determine the exact requirements for the final data handoff*

# Planning & Design

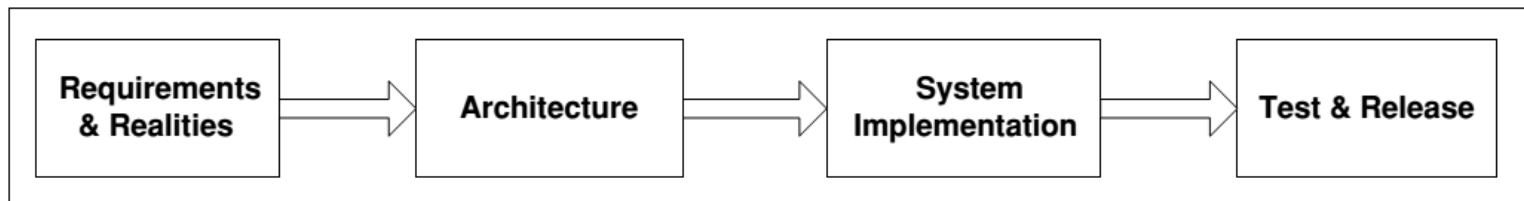
## Planning & Design thread

The second step in this thread is the **architecture** step.

*big decisions about the way we are going to build our ETL system*

*These decisions include:*

- Hand-coded versus ETL vendor tool
- Batch versus streaming data flow
- Horizontal versus vertical task dependency
- Scheduler automation
- Exception handling
- Quality handling
- Recovery and restart
- Metadata
- Security





# Architecture

## Architecture

### ETL Tool versus Hand Coding

*Buy a Tool Suite or Roll Your Own?*

### Using Proven Technology

*When it comes to building a data warehouse, many initial costs are involved - dedicated servers, database licenses, consultants and various other costs, etc.*

*in the long run that purchasing an ETL tool actually reduces the cost of building and maintaining your data warehouse*

## Architecture - Batch versus Streaming Data Flow

The standard architecture for an ETL system is based on periodic batch extracts from the source data, which then flows through the system, resulting in a batch update of the final end user tables.

*when the real-time nature of the data-warehouse load becomes sufficiently urgent, the batch approach breaks down.*

Changing from a batch to a streaming data flow changes everything

*Although we must still support the fundamental data flow steps of extract, clean, conform, and deliver, each of these steps must be modified for record-at-a-time processing*

*For instance, the basic numeric measures of a sales transaction with a new customer can arrive before the description of the customer arrives*

## Architecture - Scheduler Automation

A related architectural decision is how deeply to control your overall ETL system with automated scheduler technology

### *At one extreme*

- all jobs are triggered manually by a human operator

- For instance typing at a command line

### *At the other extreme*

- a master scheduler tool manages all the jobs

- understands whether jobs have run successfully, waits for various system statuses to be satisfied

- handles communication with human supervisors

- In case of emergency alerts and job flow status reporting

## Architecture - Exception Handling

should not be a random series of little ad-hoc alerts and comments placed in files

*should be a system-wide, uniform mechanism for reporting all instances of exceptions thrown by ETL processes into a single database*

with the name of the process, the time of the exception, its initially diagnosed severity, the action subsequently taken,  
and the ultimate resolution status of the exception.

every job needs to be architected to write these exception-reporting records into the database

## Architecture - Quality Handling

Similarly, you should decide on a common response to quality issues that arise while processing the data.

*In addition to triggering an exception reporting record, all quality problems need to generate an audit record attached to the final dimension or fact data.*

Corrupted or suspected data needs to be handled with a small number of uniform responses, such as

*filling in missing text data with a question mark*

*or supplying least biased estimators of numeric values that exist but were corrupted before delivery to the data warehouse.*

## Architecture - Recovery and Restart

From the start, you need to build your ETL system around the ability to recover from abnormal ending of a job and restart.

ETL jobs need to be impervious (immune) to incorrect multiple updating

*For instance, a job that subtracts a particular brand sales result from an overall product category should not be allowed to run twice.*

Every ETL job will sooner or later either

*terminate abnormally*

*or be mistakenly run more than once*

## Architecture - Metadata

Metadata from DBMS system tables and from schema design tools is easy to capture

*probably composes only 25 percent of the metadata you need to understand and control the system*

*another 25 percent of the metadata is generated by the cleaning step*

But the biggest metadata challenge for the ETL team is where and how to store process-flow information.

*An important but unglamorous advantage of ETL tool suites is that they maintain this process-flow metadata automatically.*

*If you are hand coding your ETL system, you need to implement your own central repository of process flow metadata.*

# Planning & Design

## Planning & Design thread

The third step is system **implementation**.

*spent some quality time on the previous two steps before charging into the implementation!*

This step includes:

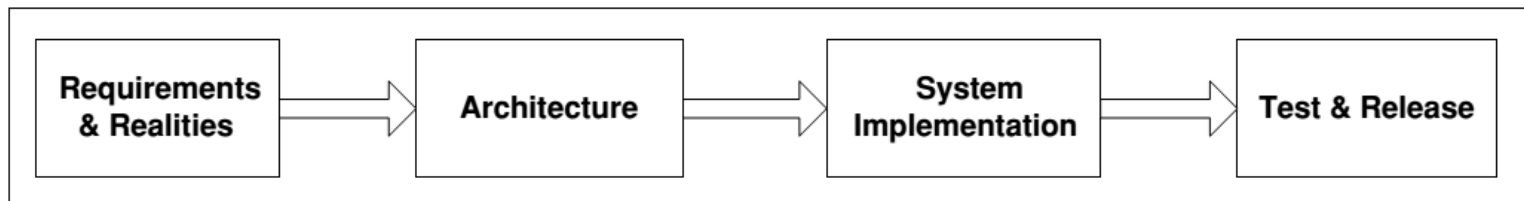
*Hardware*

*Software*

*Coding practices*

*Documentation practices*

*Specific quality checks*





# Planning & Design

## Planning & Design thread

The final step - **test and release**

*is as important as the more tangible designs of the preceding two steps*

Test and release includes the design of the:

*Development system*

*Test systems*

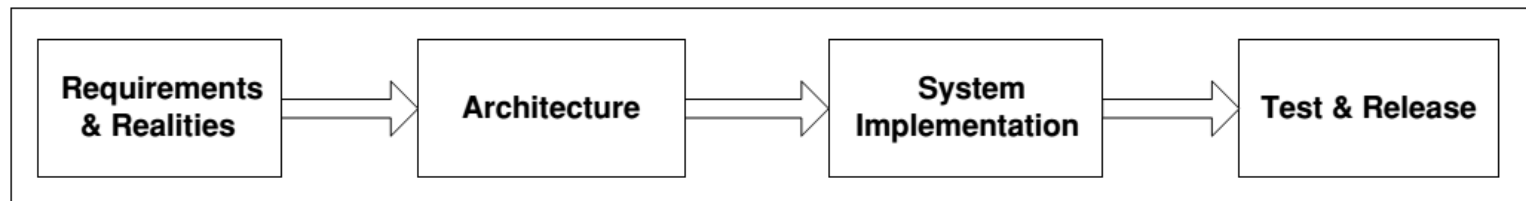
*Production systems*

*Handoff procedures*

*Update propagation approach*

*System snapshoting and rollback procedures*

*Performance tuning*



# ETL system

ETL system has four major components:

Extracting.

*Gathering raw data from the source systems and usually staging the data in the ETL environment before any significant restructuring of the data takes place.*

Cleaning and conforming.

*Sending source data through a series of processing steps in the ETL system to improve the quality of the data received from the source, and merging data from two or more sources to create and enforce conformed dimensions and conformed metrics.*

Delivering.

*Physically structuring and loading the data into the presentation server's target dimensional models.*

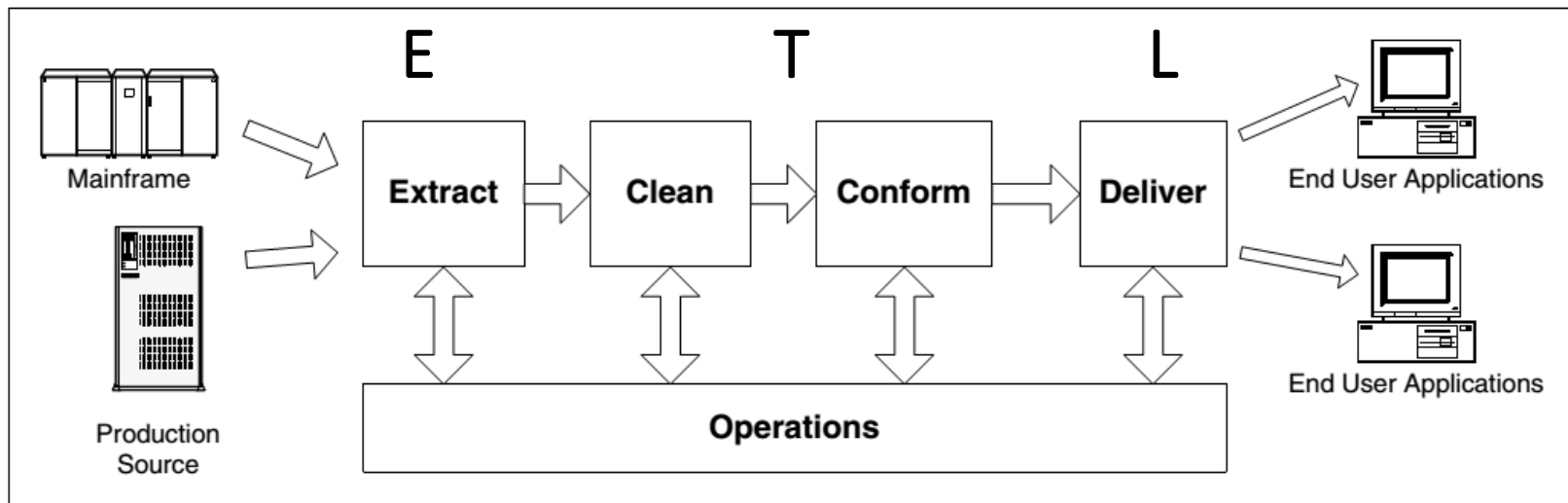
Managing.

*Managing the related systems and processes of the ETL environment in a coherent manner.*

# Data Flow

The Data Flow thread is probably more recognizable  
it is a simple generalization of the E-T-L extract-transform-load scenario

*Extract → clean → conform → deliver*



# Data Flow

## The Data Flow

The extract step includes:

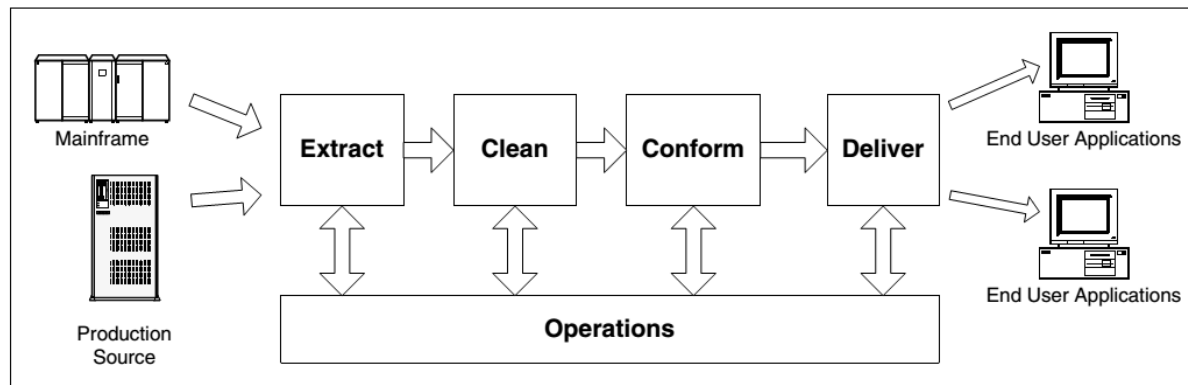
*Reading source-data models*

*Connecting to and accessing data*

*Scheduling the source system, intercepting notifications and daemons*

*Capturing changed data*

*Staging the extracted data to disk*



# Data Flow

## The Data Flow

The clean step involves:

*Enforcing column properties*

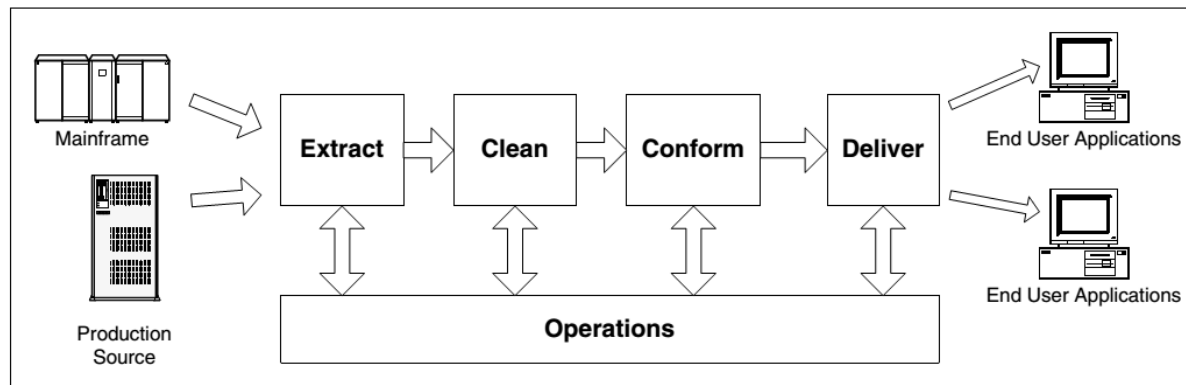
*Enforcing structure*

*Enforcing data and value rules*

*Enforcing complex business rules*

*Building a metadata foundation to describe data quality*

*Staging the cleaned data to disk*



# Data Flow

## The Data Flow

Clean step is followed closely by the conform step, which includes:

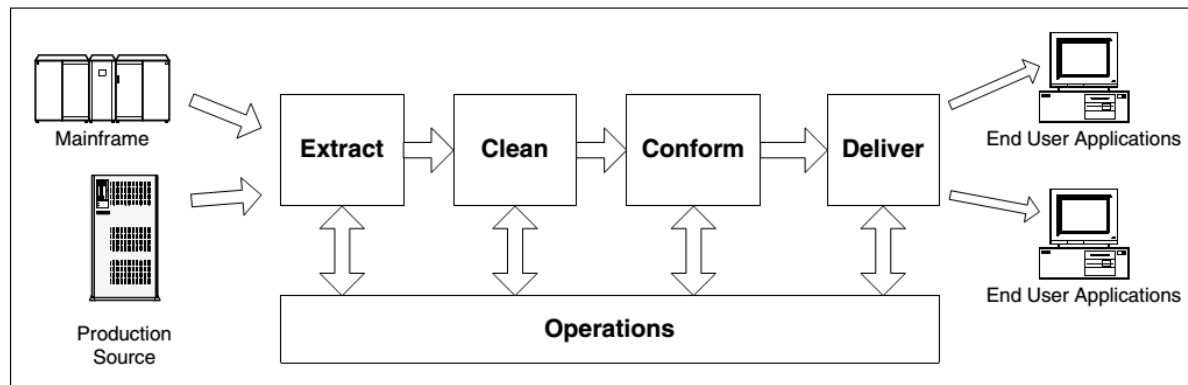
*Conforming business labels (in dimensions)*

*Conforming business metrics and performance indicators (in fact tables)*

*Deduplicating*

*Internationalizing*

*Staging the conformed data to disk*



# Data Flow

## The Data Flow

Finally deliver of data to the end-user application.

Data delivery from the ETL system includes:

*Loading flat and snowflaked dimensions (Loading subdimensions)*

*Generating time dimensions*

*Conforming dimensions and conforming facts*

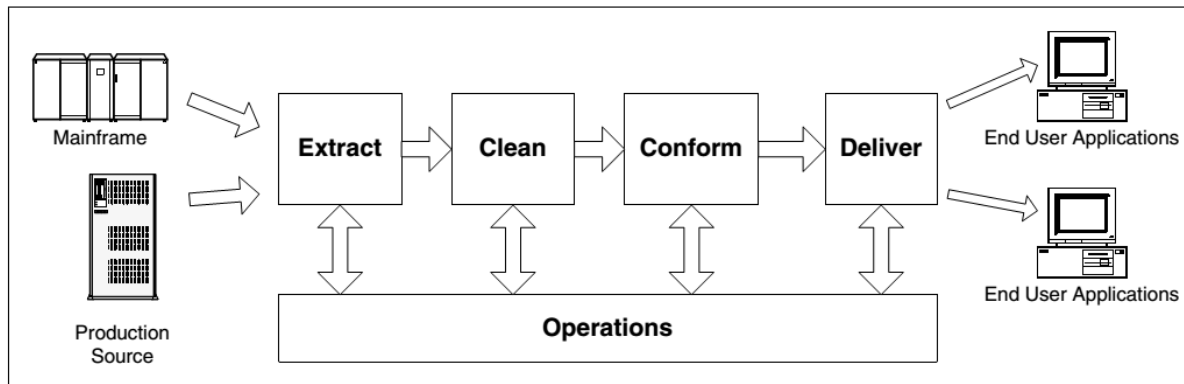
*Loading text facts in dimensions*

*Running the surrogate key pipeline for fact tables*

*Loading three fundamental fact table grains*

*Loading and updating aggregations*

*Staging the delivered data to disk*



Kimball R., Ross M.

The Data Warehouse Toolkit: The Definitive Guide to  
Dimensional Modeling, 3rd Edition

Wiley Publishing 2013

Kimball R., Caserta J..

The Data Warehouse ETL Toolkit.

Wiley Publishing 2004

Inmon W.,

Building the Data Warehouse,

John Wiley & Sons, New York 2002

Claudia Imhoff, Nicholas Galletto, Jonathan G.  
Geiger,

Mastering Data Warehouse Design - Relational and  
Dimensional Techniques,

Wiley Publishing, Inc., 2003

Short overview videos:

<https://www.youtube.com/watch?v=OW5OgsLpDCQ>

<https://www.youtube.com/watch?v=PXjSoMdFoJg>

<https://www.youtube.com/watch?v=ppKIIAsNqMs>

<https://www.youtube.com/watch?v=hKv70zftW-Y>

<https://www.youtube.com/watch?v=OSr9oKXApMU>

<https://www.youtube.com/watch?v=sLhInuwdwcc>

# Bibliography

SOURCES